Universidad Nacional de Luján

PC-SMOTE: Una variante de SMOTE controlada por percentiles

Tesina de grado presentada para optar al título de
Licenciado en Sistemas de Información

Juan Natello

Director: Santiago Banchero

2026

RESUMEN

El aprendizaje automático supervisado enfrenta un desafío persistente frente a los conjuntos de datos con desbalance de clases, una condición que, al interactuar con el solapamiento estructural, degrada severamente la capacidad predictiva de los algoritmos sobre la clase minoritaria de interés. Si bien técnicas de sobremuestreo a nivel de datos, como SMOTE, han mitigado este problema mediante la interpolación lineal, su naturaleza ciega y el uso de parámetros estáticos a menudo generan ruido y sobregeneralización al invadir el espacio topológico de la clase mayoritaria.

A diferencia de los métodos tradicionales, esta investigación introduce **PC-SMOTE** (Percentile-Controlled SMOTE), un algoritmo de doble acción fundamentado en la autocalibración geométrica. En espacios seguros y dispersos, actúa como un generador dinámico de datos sintéticos, pero en escenarios de solapamiento extremo y alto ruido, actúa intencionalmente como un **escudo protector topológico** que frena la interpolación ciega. El algoritmo utiliza percentiles para evaluar la morfología de cada clase y filtros de pureza sustentados en entropía, dotando al método de la capacidad de actuar como un **sensor de inviabilidad topológica** que detiene la generación de ruido perjudicial para el modelo.

La metodología fue validada de manera experimental utilizando un modelo Random Forest sobre diversos conjuntos de datos reales con asimetría severa (como Shuttle, US Crime, Telco Churn y Predict Faults). Los resultados empíricos demuestran que PC-SMOTE logra adaptarse de forma inteligente a la morfología de las distintas clases: en entornos con separabilidad local mejora consistentemente el rendimiento predictivo, mientras que en escenarios de extremo solapamiento estructural (como el mantenimiento industrial) restringe intencionalmente la generación de muestras. Esta negación táctica evita la inyección de alucinaciones estadísticas o "máquinas fantasma" en el modelo, mitigando proactivamente el riesgo de costosas falsas alarmas operativas en entornos de producción reales.

Palabras claves: Aprendizaje Automático, Desbalance de Clases, Solapamiento Estructural, Sobremuestreo, SMOTE, Topología de Datos, Percentiles.

AGRADECIMIENTOS

Quiero agradecer a mi director de tesis Santiago Banchero por su dedicación y su guía constante a lo largo de todo el desarrollo de este trabajo, haciendo también extensivo este reconocimiento a mi esposa por su acompañamiento durante toda esta etapa, ya que su respaldo fue un pilar fundamental para poder transitar este exigente camino y alcanzar finalmente esta meta en mi carrera profesional

A mis padres, por los valores inculcados y el ejemplo de esfuerzo.
A mis hijos Giovanna y Dante, por ser mi mayor inspiración
y el motivo de cada paso que doy hacia el futuro.

Índice general

Resumen I

Índice VIII

1.. INTRODUCCIÓN Y OBJETIVOS 1
1.1. Introducción 1
1.2. Motivación 2
1.2.1. El algoritmo como sensor de inviabilidad 2
1.3. Definición del Problema 3
1.4. Objetivos 3
1.4.1. Objetivo General 4
1.4.2. Objetivos Específicos 4
1.4.3. Estructura de la Tesis 4

2.. MARCO TEORICO 7
2.1. Aprendizaje Automático y Clasificación 7
2.2. El Problema del Desbalance de Clases 7
2.3. Técnicas de Preprocesamiento: Sobremuestreo 8
2.3.1. Evolución Algorítmica: Mejoras Internas y Sensibilidad Espacial . . 8
2.3.2. Enfoques Híbridos: Integración Externa y Filtrado de Ruido 9
2.4. Conceptos de Topología en Datos 9
2.5. Métricas de Evaluación 10
2.5.1. Precisión y Exhaustividad 10
2.5.2. F1-score 10
2.5.3. F1 macro-average 10

3.. METODOLOGÍA 13
3.1. Algoritmo Propuesto: PC-SMOTE 13
3.2. Fase 1 (auto-calibración de percentiles) 14
3.2.1. **Límites geométricos y alta dimensionalidad** 16
3.3. Fase 2 (Filtros de pureza/ densidad y riesgo) 16
3.3.1. La paradoja de la medición: el anclaje estático en un espacio dinámico 16
3.3.2. El algoritmo como sensor de inviabilidad 18
3.4. Fase 3 (Etapa de Generación adaptativa) 19
3.5. Pipeline de sobremuestreo 19
3.5.1. Descripción de los Conjuntos de Datos 21
3.6. Pipeline de entrenamiento y testeo 21

4.. RESULTADOS 25
4.1. Análisis de la auto-calibración Geométrica 25
4.2. Análisis de Radios 26
4.3. Dinámica de Filtrado y Comportamiento adaptativo de PCSMOTE 27
4.4. Análisis de resultados de la implementación de PCSMOTE 35

5.. CONCLUSIONES Y TRABAJOS FUTUROS 41
5.1. Conclusiones 41
5.2. Discusión 41
5.3. Trabajos Futuros 41

6.. Apéndice: Código Fuente 43
6.1. Algoritmo PC-SMOTE: Filtros de Selección de Semillas 43
6.1.1. Fase 1 – Filtro de Pureza 43
6.1.2. Fase 2 – Filtro de Densidad 44
6.1.3. Fase 3 – Filtro de Riesgo 45
6.1.4. Combinación de filtros y generación de muestras sintéticas 45
6.2. Pipeline de Sobremuestreo 46
6.2.1. Sobremuestreo con técnicas clásicas 46
6.2.2. Sobremuestreo con PC-SMOTE 47
6.3. Pipeline de Entrenamiento y Evaluación 48
6.3.1. Búsqueda de hiperparámetros y evaluación en test 48
6.4. Limpieza de Outliers con Isolation Forest 49

Bibliografía 51

1. INTRODUCCIÓN Y OBJETIVOS

El desbalance de clases constituye uno de los desafíos más persistentes y complejos en la construcción de modelos de aprendizaje automático supervisado, dado que esta condición puede afectar significativamente la capacidad de generalización de los algoritmos y comprometer su utilidad práctica en aplicaciones reales al provocar que el modelo tienda a sesgarse hacia la clase mayoritaria, disminuyendo drásticamente la efectividad para identificar la clase minoritaria [1, 14]. Esta vulnerabilidad resulta especialmente crítica en contextos donde las instancias minoritarias, aunque resulten escasas, revisten una altísima importancia analítica o social, tal como ocurre habitualmente en escenarios de diagnóstico médico o en la clasificación de movimientos de satélites.

Un ejemplo paradigmático de esta problemática es el dataset *Shuttle*, donde la detección de estados críticos en transbordadores espaciales se ve dificultada por el predominio masivo de condiciones de vuelo normales sobre las anomalías. Frente a estos escenarios, la mayoría de los modelos tienden a favorecer a la clase mayoritaria, exhibiendo bajos niveles de sensibilidad ante eventos infrecuentes, lo cual genera una pérdida sustantiva de información relevante.

Es por ello que las técnicas de sobremuestreo sintético han demostrado ser una de las estrategias más efectivas para mitigar dicho desbalance sin necesidad de descartar datos. Sin embargo, el desafío analítico contemporáneo no radica únicamente en la capacidad de generar nueva información, sino en la inteligencia algorítmica para saber cuándo detenerse. En este contexto, el presente trabajo se propone introducir una nueva técnica de sobremuestreo de **doble acción**, diseñada no solo para generar muestras en vecindarios seguros, sino fundamentalmente para actuar como un **escudo protector topológico** frente a escenarios donde la interpolación a ciegas resultaría destructiva para el modelo.

1.1. Introducción

Ante el escenario de desbalance previamente descrito, la respuesta convencional de la literatura ha sido el desarrollo de técnicas de preprocesamiento, entre las que destaca SMOTE como el estándar fundacional para la generación de muestras sintéticas. Si bien esta técnica permitió un avance significativo al proponer una interpolación lineal entre vecinos cercanos, su implementación original asume una homogeneidad en la distribución de los datos que rara vez se encuentra en problemas complejos. En la práctica, la eficacia de estos métodos depende críticamente de la configuración de hiperparámetros que, por lo general, se definen de manera estática o empírica.

Esta limitación técnica sugiere que aún existen enfoques inexplorados en lo que respecta a la sensibilidad del algoritmo frente a la diversidad topológica de los conjuntos de datos. Es necesario, por tanto, profundizar en mecanismos que permitan una comprensión más profunda de la geometría local de las muestras. Al entender que el comportamiento de los datos no es uniforme, se hace evidente la oportunidad de investigar soluciones que incorporen criterios de autocalibración, sentando las bases para el análisis de la adaptabilidad y el uso de percentiles como herramientas de control.

Desde una perspectiva operativa, la rigidez de estos parámetros tradicionales provoca que, en escenarios reales de alta complejidad —como el mantenimiento predictivo industrial o la detección de abandono de clientes (churn)—, los algoritmos clásicos operen a ciegas e inyecten ruido estructural. Al forzar la creación de datos en zonas altamente

solapadas, se obliga al modelo a memorizar patrones irreales o "fantasmas", derivando invariablemente en falsas alarmas y pérdidas económicas. Por consiguiente, la necesidad de una técnica de autocalibración geométrica trasciende la mera mejora estadística; se convierte en un requisito ineludible para garantizar la viabilidad y seguridad de las predicciones en el mundo real.

1.2. Motivación

Si bien el sobremuestreo es un terreno ampliamente explorado, persisten aristas de investigación críticas relacionadas con la adaptabilidad de los algoritmos a la diversidad topológica de los datos. Recientes investigaciones señalan que el éxito de los métodos de remuestreo depende en gran medida de su capacidad para identificar adaptativamente regiones problemáticas e implementar protocolos personalizados [1, p. 14]. Esto motivó principalmente la necesidad de dotar a las técnicas de generación de muestras sintéticas de una capacidad de autocalibración que responda a la geometría específica de cada conjunto de datos, donde el uso de percentiles para definir la noción de cercanía y vecindad pueda dar respuesta a la rigidez de los parámetros fijos en métodos tradicionales.

Una premisa fundamental que impulsa esta investigación es el reconocimiento de que *no todas las regiones minoritarias deben ser sobremuestreadas*. En escenarios de extremo solapamiento, forzar la interpolación de muestras sintéticas resulta contraproducente y destruye las fronteras de decisión del clasificador. Por lo tanto, se buscó desarrollar una herramienta que no solo equilibre las clases, sino que actúe como un **sensor de inviabilidad topológica**. De esta forma, el algoritmo propuesto no solo está diseñado para generar datos sintéticos de alta calidad en vecindarios seguros, sino que posee la capacidad inherente de restringir o detener la síntesis de muestras cuando detecta que la integridad estructural del espacio es crítica o altamente ruidosa.

De esta forma se busco desarrollar una herramienta que no solo equilibre las clases, sino que también actúe como un filtro inteligente de la estructura espacial, permitiendo generar datos sintéticos de mayor calidad informativa y reducir el ruido en las fronteras de decisión de los clasificadores.

1.2.1. El algoritmo como sensor de inviabilidad

Una premisa matemática fundamental que impulsa esta investigación es el reconocimiento de que saber cuándo no generar datos es un avance científico tan válido e importante como saber cómo generarlos, ya que en escenarios de extremo solapamiento topológico, equilibrar las clases a ciegas forzando la interpolación resulta contraproducente, lo cual termina por destruir irremediablemente las fronteras de decisión del clasificador subyacente.

Para dimensionar la gravedad de este problema desde el primer momento, basta con imaginar el mantenimiento predictivo de maquinaria pesada industrial al borde de una falla catastrófica, donde los parámetros térmicos y de vibración son casi idénticos a los de una máquina en perfecto estado, lo que implica que si un algoritmo clásico fuerza la generación de datos interpolando puntos en medio de ese caos topológico, estará inyectando alucinaciones estadísticas y creando el perfil de una "máquina fantasma"que la física real no soporta, provocando inevitablemente que, cuando el modelo aprenda estos patrones espurios, la planta industrial comience a recibir cientos de falsas alarmas capaces de detener líneas de producción millonarias.

Frente a este riesgo inminente, surge la razón principal por la que se buscó desarrollar una herramienta de doble acción que no solo equilibre las clases, sino que actúe valien-

temente como un sensor de inviabilidad topológica, dotando al algoritmo propuesto de la capacidad inherente de operar como un freno inteligente para detener por completo la síntesis de muestras cuando la entropía del vecindario supera los umbrales de seguridad, protegiendo así la integridad absoluta del modelo y previniendo desastres financieros en el mundo real.

1.3. Definición del Problema

Frente a este contexto, el principal inconveniente a resolver es la sinergia perjudicial que se produce entre el desbalance de clases y el solapamiento estructural (class overlap). Estudios previos han demostrado que la asimetría en la distribución de datos por sí sola no obstaculiza el aprendizaje si las clases son fácilmente separables; sin embargo, la presencia del desbalance exacerba factores de dificultad como el solapamiento y los pequeños disyuntos (small disjuncts), elevando la complejidad del problema [1, 5]. Esto produce en muchos datasets reales que las escasas muestras de la clase minoritaria se encuentren inmersas en regiones dominadas por la clase mayoritaria, degradando sustancialmente el rendimiento de los modelos de aprendizaje [3, 22].

Para ilustrar visualmente el peligro de este solapamiento extremo en una realidad de alto riesgo, basta observar el escenario del mantenimiento predictivo industrial (evaluado empíricamente en este trabajo mediante el dataset *Predict Faults*). En este entorno, las maquinarias que están a milisegundos de una falla catastrófica operan bajo parámetros térmicos y físicos casi idénticos a los de las máquinas que funcionan en perfecto estado. En consecuencia, el espacio topológico se encuentra brutalmente solapado y la frontera que separa ambas clases es imperceptible.

En este contexto de extrema complejidad, las técnicas de sobremuestreo actuales suelen interpolar muestras a ciegas, sin considerar adecuadamente si la vecindad local es estructuralmente segura. Al forzar la generación de datos sintéticos en una zona tan inestable como la descrita, estos algoritmos provocan la invasión del territorio de la clase dominante e inyectan estática de manera intencional en el modelo. Esto lleva a los clasificadores a memorizar patrones de riesgo espurios e inventar perfiles de fallas irreales que, en la práctica, generarían falsas alarmas capaces de detener líneas de producción enteras con costos astronómicos.

Por lo tanto, el problema central radica en la falta de mecanismos dinámicos y autocalibrados que permitan ajustar los umbrales de densidad, riesgo y pureza de forma automática, planteando asi el desarrollo de una técnica que garantice que la generación de muestras sintéticas se realice únicamente en zonas seguras y que posea la capacidad de detenerse ante entornos inviables para preservar la integridad topológica del dataset original.

1.4. Objetivos

Por lo anteriormente dicho, la presente investigación se orienta a contribuir al corpus académico mediante una propuesta que redefine el proceso de interpolación sintética convencional. El enfoque adoptado propone un cambio de paradigma en el tratamiento del sobremuestreo, desplazando el eje desde parámetros estáticos hacia una metodología basada en la adaptabilidad topológica de los datos, a traves del uso de percentiles.

1.4.1. Objetivo General

En este sentido, se procede a establecer el eje conductor de este trabajo, el cual se define de la siguiente forma:

- Desarrollar y validar un nuevo algoritmo de sobremuestreo, denominado PCSMOTE, basado en la autocalibración de hiperparámetros topológicos para mejorar el rendimiento de los modelos de clasificación en entornos de minería de datos con clases desbalanceadas.

1.4.2. Objetivos Específicos

Seguidamente y para dar cumplimiento al propósito central de la investigación, se enumeran los pasos necesarios para la concrecion de dicho objetivo:

- **Sintetizar** el marco teorico y las limitaciones de los metodos actuales de sobremuestreo.
- **Formalizar** el funcionamiento interno de PCSMOTE (paso a paso y base matematica)
- **Implementar** el algoritmo en un entorno de desarrollo para su reproductibilidad.
- **Demostrar** que en escenarios de solapamiento extremo y alto ruido, el rechazo a generar datos sintéticos (activación del sensor de inviabilidad) no constituye una falla del modelo, sino una acción topológica deliberada y superior para proteger la eficacia del clasificador.
- **Validar** estadisticamente la tecnica mediante pruebas experimentales de datasets con alta disparidad.

1.4.3. Estructura de la Tesis

Para asegurar una correcta comprensión y guiar al lector a través de la evolución conceptual y técnica de esta investigación, el presente trabajo se ha organizado en cinco capítulos fundamentales que articulan la propuesta algorítmica con su validación experimental y su verdadero impacto en el mundo real. A modo de punto de partida, el primer capítulo introduce la problemática del desbalance de clases y su sinergia perjudicial con el solapamiento estructural, planteando la imperiosa motivación de abandonar la interpolación a ciegas para evitar catástrofes predictivas y definiendo los objetivos que guían el desarrollo de la solución propuesta.

Sentadas estas bases, el segundo capítulo establece los fundamentos teóricos y matemáticos que sustentan el trabajo, donde se analiza la evolución histórica de las técnicas de sobremuestreo —desde el SMOTE clásico hasta sus variantes espaciales— para evidenciar la falla intrínseca de los parámetros estáticos, lo cual justifica de manera natural la adopción de la autocalibración geométrica mediante percentiles como el siguiente paso evolutivo en el estudio de la topología de datos.

A continuación, el tercer capítulo se adentra en la descripción técnica y arquitectónica de la técnica PC-SMOTE, detallando meticulosamente sus módulos funcionales para explicar cómo la flexibilidad volumétrica y los rigurosos filtros de entropía le otorgan al método la capacidad única de funcionar como un **sensor de inviabilidad topológica**, ilustrando a su vez el proceso lógico y el flujo de los datos que abarca desde el remuestreo hasta el entrenamiento y testeo del modelo de aprendizaje seleccionado.

Posteriormente, el cuarto capítulo detalla los experimentos realizados y la discusión de sus resultados. Se comparan las métricas obtenidas con las de métodos clásicos, validando que la técnica propuesta **minimiza la introducción de ruido**. Esto se traduce en una mejora práctica al proteger la frontera de decisión, confirmando la utilidad del algoritmo desarrollado.

Finalmente, el quinto capítulo presenta las conclusiones generales de la investigación, donde se reflexiona sobre las ventajas y limitaciones del modelo en función de los experimentos realizados, cerrando la tesis con la propuesta de posibles líneas de investigación y trabajos futuros orientados a la optimización paramétrica y a la extensión de esta técnica de autocalibración topológica hacia nuevos dominios de aplicación.

2. MARCO TEORICO

En el presente capítulo, se establecen las bases teóricas y conceptuales sobre las cuales esta investigación está sostenida. Inicialmente, se abordarán los fundamentos del aprendizaje automático y la clasificación supervisada, para luego profundizar en el problema del desbalance de clases y cómo el solapamiento estructural exacerba su dificultad. Posteriormente, se describirán las técnicas de preprocesamiento, haciendo especial énfasis en el sobremuestreo y en el método fundacional SMOTE. Finalmente, se introducirán los conceptos de topología en datos y la utilidad fundamental de los percentiles en la minería de datos como mecanismo de autocalibración para adaptar radios de vecindad, constituyendo el núcleo teórico del algoritmo propuesto en este trabajo.

2.1. Aprendizaje Automático y Clasificación

El aprendizaje automático supervisado tiene como objetivo principal inducir un modelo o clasificador capaz de predecir la clase de salida para instancias previamente no vistas, buscando siempre una alta capacidad de generalización [5]. Formalmente, el proceso de clasificación implica obtener una aproximación empírica de una función desconocida, con el fin de mapear un vector de entrada multidimensional (las características o atributos de los datos) hacia valores objetivo o etiquetas de clase específicas [1]. Para lograr esto, los algoritmos extraen conocimiento a partir de un conjunto de entrenamiento compuesto por instancias cuyas etiquetas ya son conocidas por el sistema.

2.2. El Problema del Desbalance de Clases

En el ámbito de la clasificación, un conjunto de datos se considera desbalanceado cuando existe una disparidad significativa en el número de muestras que pertenecen a las diferentes clases [1]. Convencionalmente, la clase con el menor número de instancias se denomina clase minoritaria (o positiva) y suele representar el concepto de mayor interés analítico, mientras que la clase predominante se conoce como clase mayoritaria (o negativa) [1].

El principal inconveniente de esta condición es que los algoritmos de clasificación estándar asumen distribuciones de clases relativamente equilibradas y son guiados por métricas de evaluación globales, como la precisión general (accuracy). Esto provoca que el modelo tienda a sesgarse hacia la clase mayoritaria para maximizar su rendimiento, ignorando a menudo a los ejemplos de la clase minoritaria al ser incapaz de distinguirlos o al tratarlos simplemente como ruido [4].

Sin embargo, revisiones recientes en la literatura demuestran que la asimetría por sí sola no obstaculiza el proceso de aprendizaje si las clases son fácilmente y linealmente separables. El desbalance actúa en realidad como un catalizador que exacerba otros factores de dificultad intrínsecos de los datos, siendo el **solapamiento de clases (class overlap)** el fenómeno más perjudicial [1]. El solapamiento ocurre cuando existen regiones en el espacio pobladas simultáneamente por muestras de diferentes clases, lo que impide definir una frontera de decisión clara sin inducir un sobreajuste [5]. Cuando la escasez de muestras minoritarias se combina con este solapamiento, las instancias positivas quedan inmersas en regiones dominadas por la clase negativa, elevando drásticamente la complejidad del problema y degradando el rendimiento de las predicciones [1].

2.3. Técnicas de Preprocesamiento: Sobremuestreo

Para mitigar los efectos del desbalance sin modificar la naturaleza de los clasificadores, se han propuesto estrategias a nivel de datos (preprocesamiento) que alteran la distribución original mediante el remuestreo [14]. Entre estas técnicas, destaca el sobremuestreo (oversampling), cuyo objetivo es equilibrar el conjunto de datos incrementando la representación de la clase minoritaria [5].

2.3.1. Evolución Algorítmica: Mejoras Internas y Sensibilidad Espacial

Para comprender la evolución de las técnicas de sobremuestreo resulta fundamental analizar primero aquellas metodologías que abordaron el problema desde la mejora interna de la técnica, comenzando por el algoritmo fundacional SMOTE introducido por Chawla et al. [2] que propuso equilibrar las clases mediante una interpolación lineal ciega, enfoque que rápidamente demostró limitaciones espaciales que fueron atacadas por variantes tempranas como Borderline-SMOTE de Han et al. [6] que restringió la generación únicamente a las muestras ubicadas en las fronteras de decisión y ADASYN de He et al. [7] que adaptó la cantidad de datos generados basándose en la distribución de densidad de la clase mayoritaria.

Esta primera etapa evolutiva dio paso a una segunda generación de algoritmos enfocados en la restricción geométrica como Radius-SMOTE de Pradipta et al. [15] que define una región segura circular [1], sumado a otras propuestas que en lugar de atacar el problema topológico de raíz buscaron mejorar el algoritmo de sus predecesores, tal como sucede con AR-ADASYN de Park y Kim [13] que intenta perfeccionar a ADASYN incorporando inteligentemente el ángulo para lograr una distribución radial más fiel pero falla al permitir la selección de vecinos a distancias excesivamente grandes [2], ocurriendo una situación similar con el algoritmo αDBASMOTE de Feng y Li [3] que ostenta un mérito analítico más abarcativo al surgir como una evolución combinada que fusiona las características tanto de Borderline-SMOTE como de ADASYN mediante la introducción de un factor de distancia ponderada para medir la vulnerabilidad espacial [3], evidenciando sin embargo que todas estas técnicas terminan heredando una desventaja crítica al no contemplar verdaderamente la heterogeneidad topológica de los conjuntos de datos debido a su dependencia de métricas rígidas, siendo esta una vulnerabilidad geométrica que PC-SMOTE resuelve con éxito al priorizar la contención dinámica de la distancia mediante percentiles antes de aplicar cualquier otra métrica de generación.

A estas innovaciones se sumaron arquitecturas estructurales de gran valor como OCH-SMOTE de Wang y Awang [21], el cual perfecciona la selección global forzando la interpolación hacia el centro de gravedad de la clase minoritaria bajo la asunción de que dicha trayectoria es segura sin evaluar exhaustivamente si atraviesa vecindarios dominados por la clase mayoritaria, particularidad conceptual que comparte con el algoritmo SOMM propuesto por Khorshidi y Aickelin [10], que busca alejar las instancias sintéticas de la zona dominante mediante vectores direccionales operando de manera reactiva al reubicar una muestra que ya fue generada en lugar de evaluar preventivamente la seguridad del entorno.

Esta oportunidad de mejora proactiva se observa también en estimadores contemporáneos de notable complejidad como LD-SMOTE de Lyu et al. [11] que estima rigurosamente la densidad local aunque deja en segundo plano la evaluación de la pureza, y VS-SMOTE de Qiu et al. [16] que se enfoca exitosamente en la pureza de los subespacios pero sin integrar un análisis profundo de densidad, evidenciando áreas de complementariedad que motivan el diseño de PC-SMOTE como una solución integral alternativa, ya que

al amalgamar los filtros de densidad, riesgo y pureza mediante percentiles autocalibrados, esta arquitectura busca dictaminar la inviabilidad de la región de forma preventiva, aplicando una negación táctica para frenar la generación antes de introducir ruido adicional y enriqueciendo así los significativos esfuerzos previos orientados a redirigir o reubicar las muestras en el espacio de datos.

2.3.2. Enfoques Híbridos: Integración Externa y Filtrado de Ruido

Por otro lado, la literatura también ha encarado el problema del desbalance de manera externa mediante enfoques híbridos que combinan el sobremuestreo con herramientas adicionales para filtrar el ruido o reestructurar el espacio antes de la clasificación, destacándose en esta rama el modelo SMOTE-MRS desarrollado por Saputra et al. [18] que integra SMOTE con agrupamiento K-Means para garantizar que las muestras sintéticas se produzcan dentro de subgrupos homogéneos, así como la propuesta SMOTE-PCA-HDBSCAN introducida por Nasaruddin et al. [12] que utiliza el análisis de componentes principales para reducir la dimensionalidad y luego aplica el clustering adaptativo HDBSCAN para detectar y eliminar el ruido de manera dinámica, a las que se suma la implementación de redes generativas antagónicas documentada por Zhou et al. [3], quienes combinaron el modelo CGAN con la distancia de Wasserstein para generar datos altamente realistas evitando la propagación de ruido.

Estas metodologías demuestran que, aunque los pipelines híbridos resultan extremadamente potentes para limpiar la estática del conjunto de datos, la incorporación de arquitecturas de agrupamiento o redes neuronales externas incrementa de manera prohibitiva el tiempo de ejecución y la dependencia de múltiples hiperparámetros adicionales, reafirmando una vez más la ventaja estratégica de la arquitectura de PC-SMOTE, la cual resuelve el desafío de la inviabilidad topológica y el filtrado de ruido de manera totalmente autónoma en una única pasada a través de sus filtros de pureza y percentiles, evitando depender de algoritmos de terceros para proteger la integridad y la velocidad del modelo.

2.4. Conceptos de Topología en Datos

Para superar las deficiencias de la interpolación ciega de SMOTE, los enfoques del estado del arte han transitado hacia una evaluación de la topología local de los datos. El análisis topológico permite categorizar a las muestras en regiones seguras (safe), de frontera (danger) o de ruido, dependiendo de la proporción y distancia de las clases presentes en su vecindario inmediato [1].

Para abordar adecuadamente la complejidad topológica resulta indispensable asimilar que la densidad y distribución de la información en un conjunto de datos real nunca se presenta de manera uniforme, lo que convierte a la utilización de hiperparámetros estáticos o radios de vecindad fijos en una estrategia altamente ineficaz al momento de intentar capturar la verdadera estructura de las fronteras de decisión. Es exactamente frente a esta rigidez analítica donde la aplicación de **percentiles** en la minería de datos adquiere una utilidad fundamental, ya que la extracción directa de estas métricas estadísticas sobre las distribuciones de distancias euclidianas, evaluadas previamente mediante métodos tradicionales como k-NN, funciona como un robusto mecanismo de **autocalibración geométrica** capaz de ajustarse inteligentemente a las constantes irregularidades del entorno espacial.

El uso de percentiles permite adaptar de manera dinámica el concepto de "radio" o "cercanía" a la morfología específica de cada clase y conjunto de datos, respondiendo orgánicamente a si un clúster de información está altamente concentrado o muy disperso.

A partir de la definición de estos radios adaptativos basados en percentiles, es posible establecer umbrales precisos de densidad local, pureza y riesgo. Esto asegura que la selección de semillas y la posterior generación de muestras sintéticas se circunscriba de manera restrictiva a entornos topológicamente seguros, mitigando la intrusión en el espacio de la clase mayoritaria y garantizando la calidad informativa de los datos generados.

2.5. Métricas de Evaluación

En esta sección se presentan las métricas utilizadas en la investigación. Comenzaremos definiendo los conceptos fundamentales de Precisión y Exhaustividad, para luego detallar el F_1-score y, finalmente, la métrica F_1-macro average [1, p. 13]. Esta última ha sido seleccionada como el indicador principal, ya que el desbalance de clases —propio de los datasets analizados— suele provocar en muchas otras métricas una falsa sensación de eficiencia o exactitud.

2.5.1. Precisión y Exhaustividad

La **Precisión** (P), también conocida como valor predictivo positivo, mide la capacidad del modelo para no etiquetar como positiva una muestra que es negativa. Una precisión baja indica un alto número de falsos positivos (FP):

$$P = \frac{TP}{TP + FP} \tag{2.1}$$

Por otro lado, la **Exhaustividad** (R, por *Recall*) o sensibilidad, mide la capacidad del modelo para detectar todas las muestras positivas dentro del conjunto de datos. Es crítica en escenarios donde omitir un caso positivo (falso negativo) tiene un alto costo:

$$R = \frac{TP}{TP + FN} \tag{2.2}$$

Donde TP representa los verdaderos positivos y FN los falsos negativos. En el desarrollo de variantes de SMOTE controladas por percentiles, el objetivo central es mejorar el *Recall* de la clase minoritaria sin degradar drásticamente la precisión.

2.5.2. F1-score

El F_1-score es la media armónica entre la precisión y la exhaustividad. Se utiliza como una métrica única que busca un equilibrio entre ambas. A diferencia de la media aritmética, la media armónica penaliza fuertemente los valores extremos; si cualquiera de las dos métricas (P o R) es baja, el F_1-score reflejará dicho rendimiento deficiente:

$$F1 = 2 \cdot \frac{P \cdot R}{P + R} \tag{2.3}$$

2.5.3. F1 macro-average

La métrica principal para evaluar los resultados de esta investigación es el **F_1-macro average**. Su elección responde a la necesidad de evaluar el rendimiento del modelo de manera equitativa ante el desbalance de datos. A diferencia del promedio simple (*micro-average*), el enfoque *macro* calcula el valor F_1 de forma independiente para cada clase y luego obtiene el promedio aritmético, otorgando el mismo peso a la clase minoritaria que a la mayoritaria.

Para definir el F_1-macro, consideramos el F_1 para cada clase i:

$$F1_i = 2 \cdot \frac{P_i \cdot R_i}{P_i + R_i} \tag{2.4}$$

Finalmente, el F_1-macro se calcula promediando los valores obtenidos para las n clases del conjunto de datos:

$$F1_{\text{macro}} = \frac{1}{n} \sum_{i=1}^{n} F1_i \tag{2.5}$$

Este procedimiento garantiza que cualquier mejora en la clasificación de la clase minoritaria se vea reflejada directamente en la métrica global, evitando que el alto desempeño en la clase predominante enmascare deficiencias del modelo en los casos de mayor interés para este estudio.

3. METODOLOGÍA

En estrecha consonancia con los fundamentos teóricos establecidos en el capítulo anterior, la presente sección describe el diseño metodológico adoptado para abordar el problema de la clasificación en conjuntos de datos con un desbalance severo. Esta investigación es de naturaleza experimental y cuantitativa, con el propósito fundamental de formalizar y evaluar la eficacia de la técnica de sobremuestreo topológico PC-SMOTE, contrastándola con estrategias de remuestreo ya establecidas en el estado del arte. La validación empírica resulta indispensable en este contexto, ya que permite aislar el efecto de la topología local de los datos y medir de forma objetiva la capacidad del nuevo algoritmo para mejorar la generalización del modelo predictivo en escenarios reales de alta complejidad.

Esta evolución metodológica resulta crucial ya que, como afirman Santos et al. [17] y respaldan revisiones recientes de la literatura, el desbalance de clases por sí solo no es el único responsable de la degradación del rendimiento de los modelos predictivos; es su sinergia con factores de dificultad intrínsecos —particularmente el solapamiento de clases (*class overlap*), los pequeños disyuntos (*small disjuncts*) y el ruido— lo que constituye el escenario estructural más perjudicial para el aprendizaje automático [1, p. 6]. Bajo estas condiciones, las técnicas tradicionales de sobremuestreo (como SMOTE) que operan mediante interpolación lineal a ciegas tienden a generar muestras sintéticas en regiones dominadas por la clase mayoritaria, incrementando el ruido y provocando un sobreajuste o sobregeneralización de las fronteras de decisión [1, p. 18]. Esto hace imperativo plantear un diseño orientado a los datos (*data-centric*) que evalúe activamente la viabilidad del espacio antes de sintetizar nueva información.

Para dar respuesta a este desafío estructural y garantizar una evaluación rigurosa, la metodología de este trabajo se articula en dos etapas principales. En primera instancia, se describe exhaustivamente la arquitectura del algoritmo propuesto (PC-SMOTE), el cual ha sido diseñado para operar en tres fases esenciales: (1) la autocalibración geométrica de percentiles para definir de manera dinámica los radios de vecindad; (2) la aplicación de filtros topológicos de pureza, densidad y riesgo para categorizar las muestras y restringir la selección de semillas a zonas seguras; y (3) la etapa de generación e interpolación adaptativa para crear muestras minoritarias mitigando la invasión del territorio opuesto.

En segunda instancia, se detalla el *pipeline* (flujo de trabajo) de entrenamiento y testeo implementado para la experimentación. Este abarca desde la preparación y limpieza inicial del conjunto de datos mediante la detección de anomalías con *Isolation Forest*, hasta la búsqueda y optimización de hiperparámetros. Finalmente, se describe el proceso de evaluación del modelo base seleccionado (Random Forest), justificando la elección de métricas de rendimiento robustas frente al desbalance —como la métrica *F1-macro*—, las cuales garantizan una medición justa y equitativa de la capacidad del clasificador para reconocer a la clase minoritaria de interés.

3.1. Algoritmo Propuesto: PC-SMOTE

Previo a detallar exhaustivamente los fundamentos de cada una de las fases que componen la arquitectura del modelo, se presenta a continuación la formalización general en pseudocódigo del algoritmo PC-SMOTE (Algoritmo 1). Esta representación permite visualizar el flujo lógico de los datos, desde la calibración geométrica de los espacios hasta la generación sintética restrictiva. La implementación completa en Python se encuentra

disponible de forma pública en el repositorio del proyecto.[1]

Algorithm 1 PC-SMOTE (Percentile-Controlled SMOTE)

Require: D: Conjunto de datos de entrenamiento ($D_{min} \cup D_{maj}$)
Require: k: Número de vecinos más cercanos a evaluar
Require: $PRdens, PRriesgo$: Percentiles base para la auto-calibración de radios
Require: $U_{dens}, U_{riesgo}, U_{pureza}$: Umbrales de aceptación topológica
Ensure: D_{new}: Conjunto de datos balanceado

// **Fase 1: Auto-calibración Geométrica**
$Distancias \leftarrow \emptyset$
for cada $x_i \in D_{min}$ **do**
    $Vecinos \leftarrow$ obtener los k vecinos más cercanos de x_i en D
    Agregar las distancias euclidianas de $Vecinos$ a $Distancias$
end for
$Radio_{dens} \leftarrow Percentil(Distancias, PRdens)$
$Radio_{riesgo} \leftarrow Percentil(Distancias, PRriesgo)$

// **Fase 2: Filtros Topológicos (Densidad, Riesgo y Pureza)**
$Semillas_Validas \leftarrow \emptyset$
for cada $x_i \in D_{min}$ **do**
    $Densidad_i \leftarrow \frac{|\{x_j \in D_{min} \text{ tal que } dist(x_i,x_j) \leq Radio_{dens}\}|}{k}$
    $Riesgo_i \leftarrow \frac{|\{x_j \in D_{maj} \text{ tal que } dist(x_i,x_j) \leq Radio_{riesgo}\}|}{k}$
    $Pureza_i \leftarrow CalcularPureza(x_i, k)$ ▷ Evaluado por proporción o entropía
    if $Densidad_i \geq U_{dens}$ **and** $Riesgo_i \leq U_{riesgo}$ **and** $Pureza_i \geq U_{pureza}$ **then**
        $Semillas_Validas \leftarrow Semillas_Validas \cup \{x_i\}$
    end if
end for

// **Fase 3: Etapa de Generación e Interpolación Adaptativa**
$N_{sinteticos} \leftarrow |D_{maj}| - |D_{min}|$
$Sinteticos \leftarrow \emptyset$
while $|Sinteticos| < N_{sinteticos}$ **and** $|Semillas_Validas| > 0$ **do**
    $x_{semilla} \leftarrow$ Seleccionar muestra aleatoria de $Semillas_Validas$
    $x_{vecino} \leftarrow$ Seleccionar vecino aleatorio de $x_{semilla}$ dentro del $Radio_{dens}$
    $\alpha \leftarrow Aleatorio(0{,}4, 0{,}6)$ ▷ Control de sobregeneralización
    $x_{nuevo} \leftarrow x_{semilla} + \alpha \times (x_{vecino} - x_{semilla})$
    $Sinteticos \leftarrow Sinteticos \cup \{x_{nuevo}\}$
end while
Retornar $D_{new} \leftarrow D \cup Sinteticos$

3.2. Fase 1 (auto-calibración de percentiles)

En esta fase se determinan los umbrales de radio por densidad y riesgo, junto con el umbral de entropía. Estos valores establecen los límites operativos de la vecindad, adaptando la escala espacial a la estructura de cada clase.

En este sentido, la auto–calibración a través de percentiles permite ajustar los umbrales

[1] Repositorio del proyecto: `https://github.com/jnatmed/etapa-analisis_pcsmote`

de radio tanto para los cálculos de densidad como de riesgo, así como el umbral de entropía, de acuerdo con el percentil configurado. Para ello, el algoritmo toma cada muestra de la clase actual y recorre sus k vecinos más cercanos utilizando un clasificador k-NN. Para cada vecino se calcula la distancia euclidiana respecto de la muestra de referencia, y al finalizar este proceso se obtienen $X \times k$ distancias totales, donde X corresponde al número de instancias de la clase evaluada.

Sobre el conjunto total de distancias se calcula el percentil definido por el parámetro de configuración. El valor resultante se interpreta como un radio adaptativo que representa el umbral por densidad y el umbral por riesgo.

> *≪La necesidad de abandonar un radio de distancia fijo en favor de percentiles dinámicos responde a realidades operativas críticas. Si consideramos el dominio de las telecomunicaciones (como en el dataset Telco Churn), un cliente que cancela su servicio influenciado por factores externos impredecibles genera una "nube de dispersion" en el espacio topológico. Un radio estático y rígido jamás podría capturar esta morfología sin absorber ruido masivo, justificando así la imperativa flexibilidad volumétrica que aporta la Fase 1 de PC-SMOTE. Para dimensionar la urgencia y el impacto operativo de esta decisión topológica, es necesario abandonar momentáneamente la abstracción matemática. En el dominio de las telecomunicaciones (como se evaluará en el dataset Telco Churn), la nube de dispersión no representa un simple agrupamiento euclidiano alejado, representa el caos humano. Son clientes reales que cancelan su servicio por factores externos impredecibles, como una mudanza repentina. Un algoritmo basado en radios fijos y estáticos capturaría esta nube y la trataría como un verdadero patrón de abandono que el modelo debe memorizar, pero al vincular el percentil de densidad dinámico con el comportamiento humano real, produce que PC-SMOTE evite replicar este ruido topológico. De este modo, la calibración matemática deja de ser un simple ajuste de hiperparámetros y se convierte en una intervención operativa crítica, evitando que las empresas desperdicien presupuesto en campañas de retención inútiles para perfiles dictados por el azar.≫*

En el paso siguiente se calculan todas las métricas de densidad, riesgo y pureza —ya sea por proporción o por entropía— para la clase actual, utilizando los umbrales previamente obtenidos.

El cálculo de estas métricas, que caracterizan el comportamiento de los casos minoritarios en un conjunto de datos desbalanceado, constituye un proceso multifacético basado en la evaluación de la vecindad de cada instancia. En primer lugar, se determina la **densidad** de cada caso minoritario. Esta se obtiene contando cuántos de sus k vecinos más cercanos pertenecen a la misma clase minoritaria y se encuentran a una distancia menor o igual al *Percentil Radio Densidad* (PRD). Una densidad elevada sugiere que la instancia está bien rodeada por ejemplos de su propia clase.

En contraste, para evaluar la vulnerabilidad o la posible ubicación en la frontera entre clases, se calcula el **riesgo** de cada caso minoritario. Este cálculo consiste en contabilizar cuántos de los k vecinos más cercanos pertenecen a la clase mayoritaria y se encuentran a una distancia menor o igual al *Percentil Radio Riesgo* (PRR). Un riesgo elevado indica que la instancia minoritaria se encuentra peligrosamente próxima a la clase dominante, lo que sugiere que podría tratarse de un caso en zona de frontera o de difícil clasificación.

La última métrica puede aplicarse de dos maneras diferentes. La primera correspon-

de a la **pureza por proporción**, que consiste en contar cuántos vecinos de la misma clase existen dentro de la vecindad definida por los k vecinos más cercanos. Este cálculo constituye una forma directa de evaluar la cohesión local de la clase minoritaria, sin imponer una restricción basada en el radio percentil, proporcionando una visión clara de la concentración local de la clase de interés.

Alternativamente, la pureza puede evaluarse mediante la **entropía**. En este caso, la métrica se calcula directamente para cada instancia minoritaria a partir de la distribución de clases presentes entre sus k vecinos más cercanos. La entropía actúa como una medida de incertidumbre o mezcla de clases en el entorno inmediato de la instancia. Un valor alto de entropía indica un entorno altamente mezclado o ruidoso, lo que sugiere que la instancia minoritaria se encuentra ubicada en una zona de solapamiento significativo entre clases.

3.2.1. Límites geométricos y alta dimensionalidad

Es imperativo reconocer una limitación topológica inherente a esta fase de autocalibración, ya que el sustento matemático para la captura de vecindades asume la efectividad de la distancia euclidiana principalmente en espacios de baja a moderada dimensionalidad, lo cual significa que, frente a conjuntos de datos masivos con cientos de características (ultradimensionales), donde el volumen del espacio geométrico se expande de manera exponencial, provoca que las distancias entre los puntos tiendan a converger hasta hacerse prácticamente equidistantes —un fenómeno conocido como la "maldición de la dimensionalidad"[16, 21]—, de modo que si la autocalibración operara directamente sobre este nivel de saturación, haría que el cálculo de los percentiles heredara irremediablemente dicho error, ajustando sus radios basándose en puro ruido, por lo que se establece formalmente que, para garantizar la seguridad topológica de PC-SMOTE en este tipo de entornos, el algoritmo requerirá ineludiblemente la aplicación previa de técnicas de reducción de dimensionalidad (como PCA) [12, 18], asegurando de esta forma que la métrica euclidiana logre conservar su verdadera capacidad discriminatoria antes de iniciar el proceso.

3.3. Fase 2 (Filtros de pureza/ densidad y riesgo)

A estos conjuntos de densidades, riesgos y pureza (proporción — riesgo) se les aplica el filtro correspondiente de densidad, riesgo y pureza, también recibidos por parámetro, quedando solo aquellas semillas que hayan superado los tres filtros.

Como demuestran enfoques recientes (Predipta et. al.[15]) filtrar las muestras minoritarias iniciales para descartar el ruido y conservar unicamente las instancias situadas en zonas estructuralmente seguras (SAFE), garantiza que la posterior generacion de sinteticos no invada el territorio de la clase mayoritaria ni provoque nuevos conflictos en las regiones de solapamiento.

3.3.1. La paradoja de la medición: el anclaje estático en un espacio dinámico

Antes de aplicar los filtros topológicos, la arquitectura de la Fase 2 debe resolver un dilema metodológico fundamental respecto al cálculo de la entropía utilizando un parámetro de vecindad (k) fijo, dado que tras argumentar exhaustivamente a favor de la flexibilidad volumétrica total en la Fase 1, la imposición de una evaluación restrictiva podría percibirse como una contradicción, por lo que para asimilar la necesidad de este aparente salto resulta útil una analogía topográfica donde, para evaluar de forma confiable la irregularidad del terreno en regiones de dimensiones drásticamente distintas que representan el radio

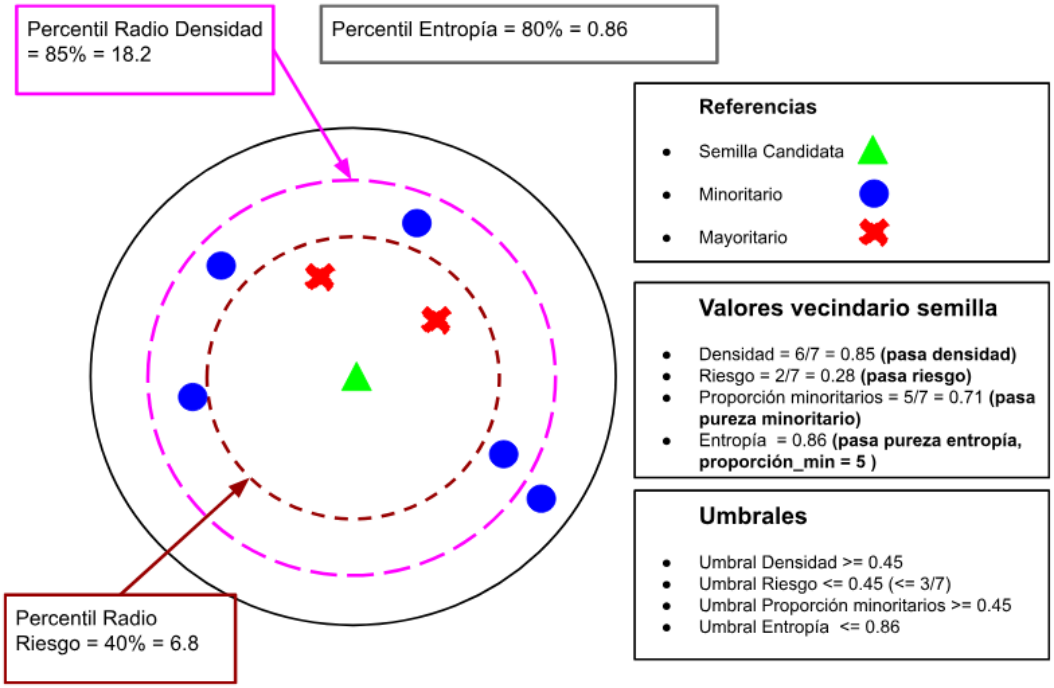

Figura 3.1: Caso Aceptacion Semilla Candidata

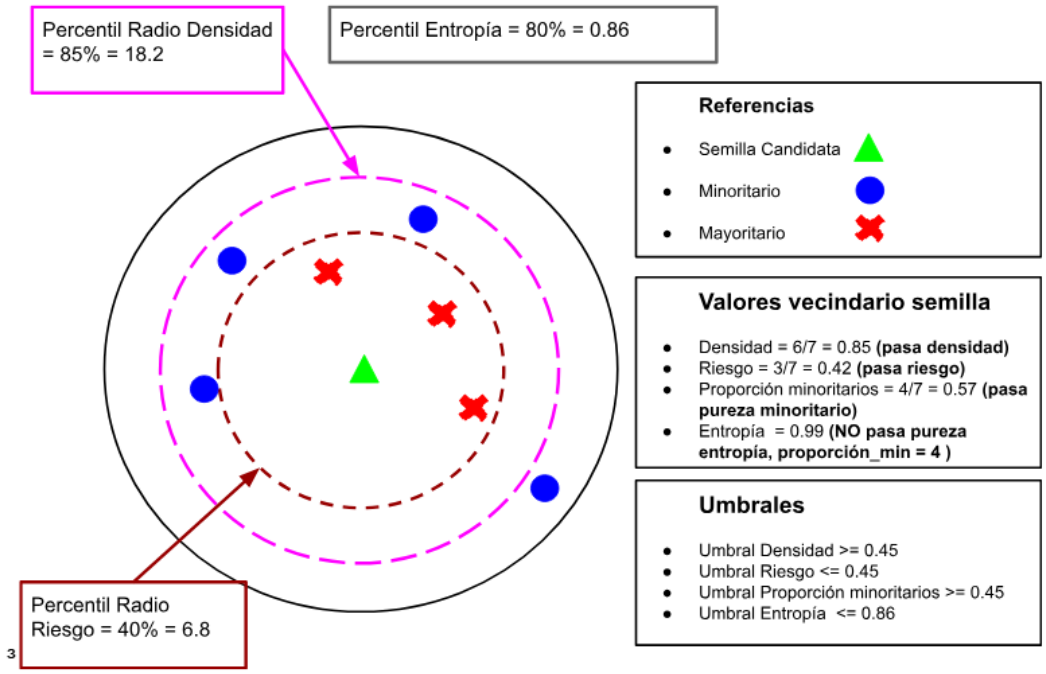

Figura 3.2: Caso Rechazo Semilla Candidata

espacial dinámico, un instrumento de sondeo requiere capturar siempre una cantidad de puntos de referencia fija e inamovible, ya que si el número de puntos evaluados variara geométricamente con cada expansión o contracción del terreno, resultaría imposible registrar un patrón estructural coherente o establecer un umbral de caos universal.

Esta decisión no constituye una regresión topológica, sino un requisito de normalización algorítmica dictado por la Teoría de la Información, debido a que para que la entropía de Shannon funcione como una lente comparativa válida a lo largo de toda la morfología del conjunto de datos requiere obligatoriamente una unidad de medida estable o denominador constante, lo que significa que si el algoritmo evaluara cincuenta vecinos en una región densa y solo tres en una dispersa las escalas de impureza se distorsionarían por completo haciendo que comparar ambas regiones sea matemáticamente espurio, siendo este el motivo exacto por el cual la presente arquitectura introduce de manera deliberada un valor constante de $k = 7$ vecinos y un umbral de tolerancia de aceptación de $0,4$ para la evaluación de la pureza, actuando así como el instrumento de anclaje indispensable que evalúa siempre a una cantidad invariable de muestras operando dentro de un radio espacial que sí respira y se adapta a la densidad local.

Tabla 3.1: Análisis de vecindarios y criterios de aceptación en PC-SMOTE basados en entropía y proporción de clase.

V. Min.	**V. May.**	**Prop. Min.**	**Entropía**	**Interpretación**	**Aceptación PC-SMOTE**
7	0	1,00	0,00	Pureza absoluta	Aceptado (Ideal)
6	1	0,86	0,59	Contaminación mínima	Aceptado
5	2	0,71	0,86	Contaminación moderada	Aceptado
4	3	0,57	0,99	Frontera (Dom. Min.)	Aceptado
3	4	0,43	0,99	Frontera (Dom. May.)	Aceptado ($t_{dom} = 0,4$)
2	5	0,29	0,86	Ruido	Descartado
1	6	0,14	0,59	Ruido	Descartado
0	7	0,00	0,00	Vecindario opuesto	Descartado

De esta forma se logra que la semilla candidata sea sometida a tres análisis topológicos, es decir que la semilla se encuentre dentro de un vecindario lo suficientemente denso, que tenga un máximo de vecinos contrarios cerca (bajo riesgo) y que el vecindario a su vez sea lo más puro posible.

El uso de los percentiles y umbrales, es regular de manera escalonada y controlada el cálculo de los vecindarios calculados por el clasificador knn, porque el mismo no analiza que tan cerca están los vecinos, solo lo busca, sin importar que tan cerca o lejos están. Con PCSMOTE, se analiza estos k vecinos encontrados por knn, mediante el análisis topológico de cada vecindario donde se define si una semilla es buena para la creación de muestras sintéticas de la misma clase.

3.3.2. El algoritmo como sensor de inviabilidad

Una premisa matemática fundamental que impulsa esta investigación es el reconocimiento de que *saber cuándo no generar datos es un avance tan válido e importante como saber cómo generarlos.* En escenarios de extremo solapamiento topológico, equilibrar las clases a ciegas forzando la interpolación resulta contraproducente y destruye las fronteras de decisión del clasificador subyacente.

Para dimensionar la gravedad de este problema en el mundo real, basta observar entornos de la industria pesada (como el mantenimiento predictivo de maquinarias), donde las máquinas que están a punto de fallar operan bajo parámetros casi idénticos a las máquinas sanas, lo que crea un ambiente brutalmente solapado, por lo que forzar a un algoritmo a inventar perfiles de fallas irreales en esa zona generará falsas alarmas que pueden detener líneas de producción enteras, costando millones de dólares a la industria.

Por lo tanto, se buscó desarrollar una herramienta de doble acción que no solo equilibre las clases, sino que actúe como un **sensor de inviabilidad topológica**. De esta forma, PC-SMOTE previene este tipo de desastres al negarse a inventar perfiles irreales. De esta forma, la tecnica resalta su capacidad inherente de actuar como un freno inteligente, restringiendo o deteniendo por completo la síntesis de muestras cuando la entropía supera los umbrales de seguridad, protegiendo así la integridad del modelo.

3.4. Fase 3 (Etapa de Generación adaptativa)

En esta fase, al momento de examinar cada una de las semillas que pasaron el filtro, se seleccionan aquellos vecinos de su misma clase de manera aleatoria, pero se usan solo aquellos que estén dentro del $PR_{\text{radio_densidad}}$, a fin de asegurar la generación de sintéticos dentro del vecindario seguro.

Además, a diferencia de *SMOTE* que trabaja con σ entre 0,1 y 0,9, *PCSMOTE* genera un σ entre 0,4 y 0,6 para procurar generar el nuevo sintético en medio de ambas muestras. La idea central es que no esté "accidentalmente" pegado a ninguna de las dos muestras, es decir, ni demasiado pegado a la semilla que superó los filtros, ni demasiado pegado al vecino con el cual va a interpolar.

Finalmente, tras estas dos verificaciones se procede a realizar el cálculo de interpolación correspondiente.

3.5. Pipeline de sobremuestreo

La Figura 3.4 ilustra el *pipeline* de sobremuestreo. El proceso comienza con la división de cada *dataset* en conjuntos de entrenamiento (*train*) y de prueba (*test*). A partir del conjunto de entrenamiento, se calculan la mediana y el rango intercuartílico, definido por el primer cuartil (Q_1, percentil 25) y el tercer cuartil (Q_3, percentil 75).

Posteriormente, tanto el conjunto de entrenamiento como el de prueba se escalan. La normalización se realiza utilizando la técnica *Robust Scaler* (ver Ecuación 3.1), que consiste en restar la mediana a cada muestra y dividir el resultado por la diferencia entre Q_3 y Q_1:

$$X_{scaled} = \frac{X - \text{mediana}}{Q_3 - Q_1} \tag{3.1}$$

El conjunto de datos de testeo se reserva inicialmente para la etapa de prueba del modelo de aprendizaje, tras lo cual el trabajo se centra exclusivamente en el conjunto de entrenamiento. El conjunto de datos de testeo se obtiene de este conjunto inicial y se utiliza para evaluar el rendimiento final del modelo.

El proceso de preparación del conjunto de entrenamiento comienza con la Limpieza de Datos, donde se aplica el algoritmo *Isolation Forest*. Este proceso se ajusta mediante umbrales de percentiles predefinidos (0 % para no aplicación, 1 % para limpieza leve, 3 % para limpieza media y 5 % para limpieza profunda) para controlar el nivel de depuración.

Una vez reducido el tamaño del conjunto de entrenamiento a causa de la limpieza, se procede al Sobremuestreo (*Oversampling*) Clásico. Para esta etapa se emplean técnicas

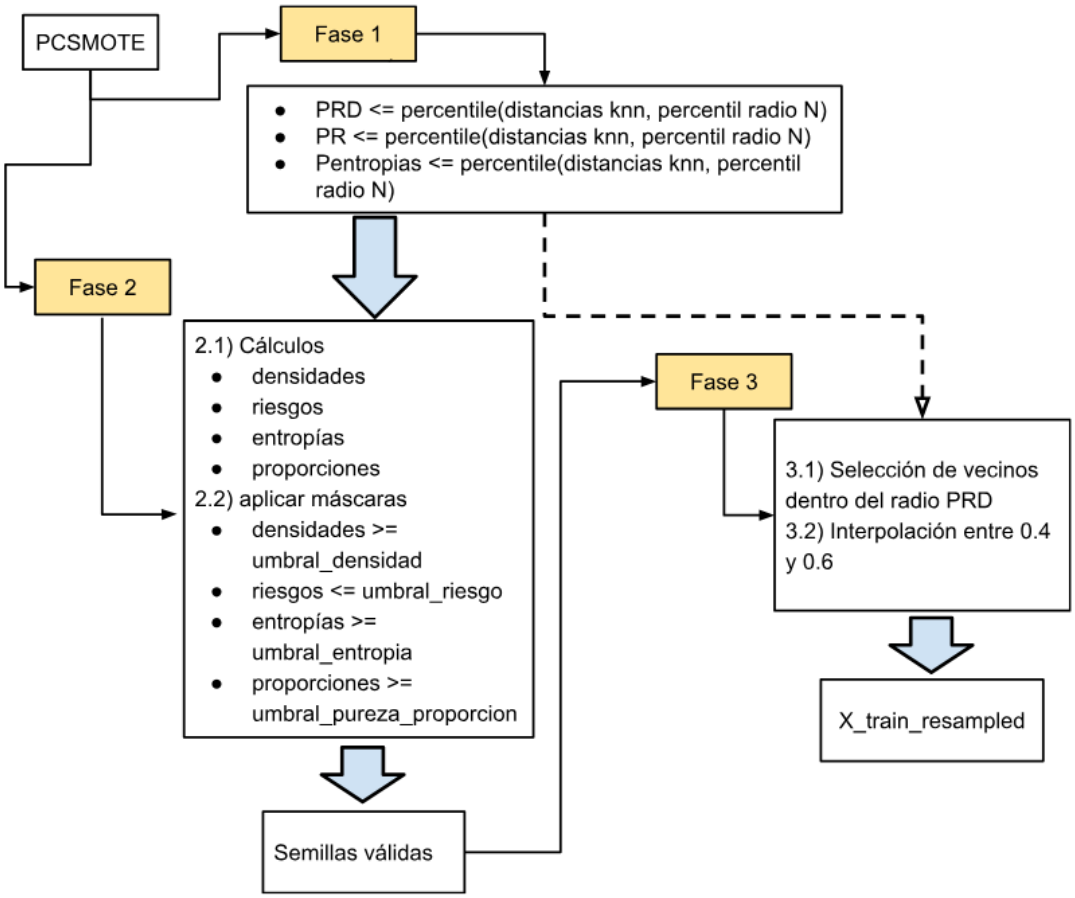

Figura 3.3: Etapas del Sobremuestreo de PCSMOTE

conocidas como *SMOTE*, *Borderline SMOTE* y *ADASYN*. Finalmente, se aplica la técnica de sobremuestreo propuesta en este trabajo: *PCSMOTE*.

Como resultado de estas etapas de preparación, se obtienen cinco versiones distintas del conjunto de entrenamiento (*train set*) para cada *dataset*: el conjunto original, y cuatro conjuntos sobremuestreados utilizando *SMOTE*, *Borderline SMOTE*, *ADASYN* y *PCSMOTE*, respectivamente. Con la obtención de estos cinco conjuntos, concluye la fase de preparación de datos, dando paso a la fase de entrenamiento y testeo del modelo de aprendizaje.

3.5.1. Descripción de los Conjuntos de Datos

Con el firme propósito de evaluar rigurosamente la capacidad de autocalibración geométrica y el comportamiento del algoritmo PC-SMOTE frente a diversos escenarios de complejidad topológica, se seleccionaron cuatro conjuntos de datos reales que presentan desafíos críticos de desbalance y solapamiento estructural.

En primer lugar, destaca el dataset Shuttle, el cual se caracteriza por un predominio masivo de las condiciones de vuelo normales frente a las anomalías. Esto genera una pérdida sustantiva de información relevante al operar sobre 9 características numéricas que describen las posiciones de los radiadores en un transbordador espacial de la NASA [19].

A esto se suma el conjunto de datos Telco Churn, que refleja un escenario comercial de telecomunicaciones donde el abandono del servicio crea nubes de dispersión altamente irregulares [8]. Asimismo, se incorpora el dataset Predict Faults, cuyo entorno de mantenimiento predictivo industrial simula fallas en maquinarias empleando parámetros físicos y térmicos casi idénticos a los de funcionamiento normal [9].

Finalmente, el conjunto US Crime aporta un nivel adicional de ruido espacial mediante 143 características demográficas y socioeconómicas originales, incluyendo niveles de ingresos y composición racial [20]. Esta colección integral permite validar cómo la técnica propuesta adapta sus radios percentiles a diversas morfologías para mejorar el rendimiento predictivo.

Tabla 3.2: Características principales de los conjuntos de datos experimentales

Dataset	**Instancias**	**Atributos**	**Distribución**	**Tipo**
Shuttle [19]	58.000	9	Maj: 45.586 Min: 12.414	Multiclase (7)
Predict Faults [9]	10.000	Múltiples	Maj: 9.652 Min: 348	Binaria
Telco Churn [8]	7.043	Múltiples	Maj: 5.174 Min: 1.869	Binaria
US Crime [20]	1.994	143	Maj: 1.844 Min: 150	Binaria

3.6. Pipeline de entrenamiento y testeo

El pipeline de Entrenamiento y Testeo se estructura en dos fases. La primera fase consiste en la **Optimización del Modelo Base**. Dentro de esta, se realiza el *Entrenamiento*

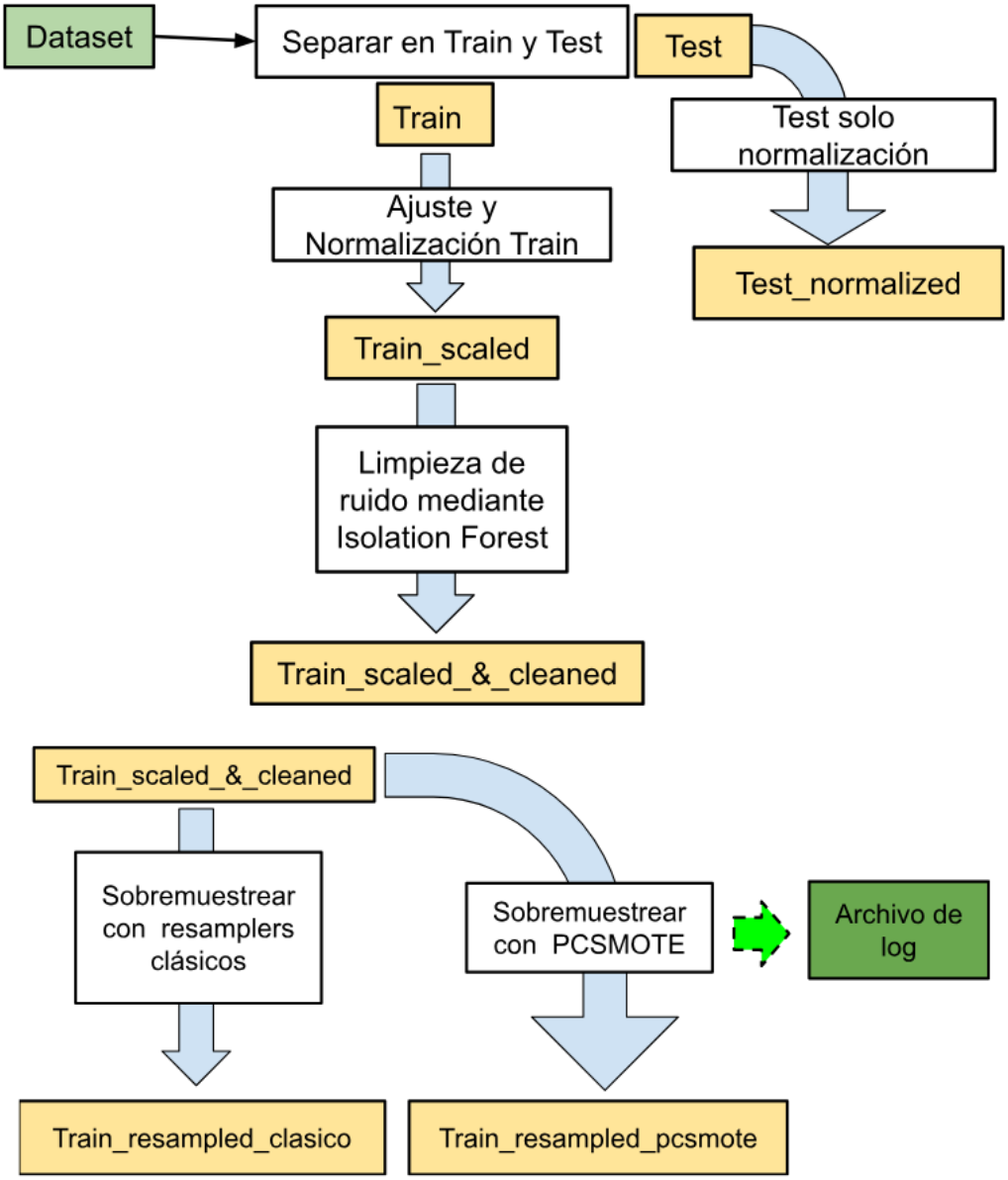

Figura 3.4: Etapas del Sobremuestreo de PCSMOTE

y Búsqueda de Hiperparámetros, empleando el conjunto de entrenamiento original para realizar una búsqueda en grilla mediante *Random Search CV*, con el fin de identificar los mejores hiperparámetros para un modelo de aprendizaje *Random Forest*.

Posteriormente, se lleva a cabo la *Evaluación del Modelo Base*. Utilizando los hiperparámetros óptimos hallados, se evalúa el rendimiento del modelo sobre el conjunto de testeo, calculando las métricas $F1_{macro}$, $F1_{weighted}$ y precisión (*accuracy*) mediante validación cruzada.

La segunda fase es la **Evaluación de Estrategias de Sobremuestreo**. En esta etapa, se realiza el *Entrenamiento y Testeo con Sobremuestreo*. Se toma un nuevo modelo *Random Forest* configurado con los mejores hiperparámetros hallados previamente. Este modelo se entrena individualmente con cada tipo de conjunto de entrenamiento sobremuestreado (producidos por *SMOTE*, *Borderline SMOTE*, *ADASYN* y *PCSMOTE*) y se evalúa con el conjunto de testeo correspondiente.

Finalmente, se procede al *Análisis Comparativo*. Se obtienen las métricas $F1_{macro}$, $F1_{weighted}$ y precisión para cada variante de sobremuestreo. Esto permite establecer un *ranking* comparativo, centrado en la métrica de interés ($F1_{macro}$), y determinar el porcentaje de mejora (Δ) de cada estrategia respecto al caso base o *baseline*.

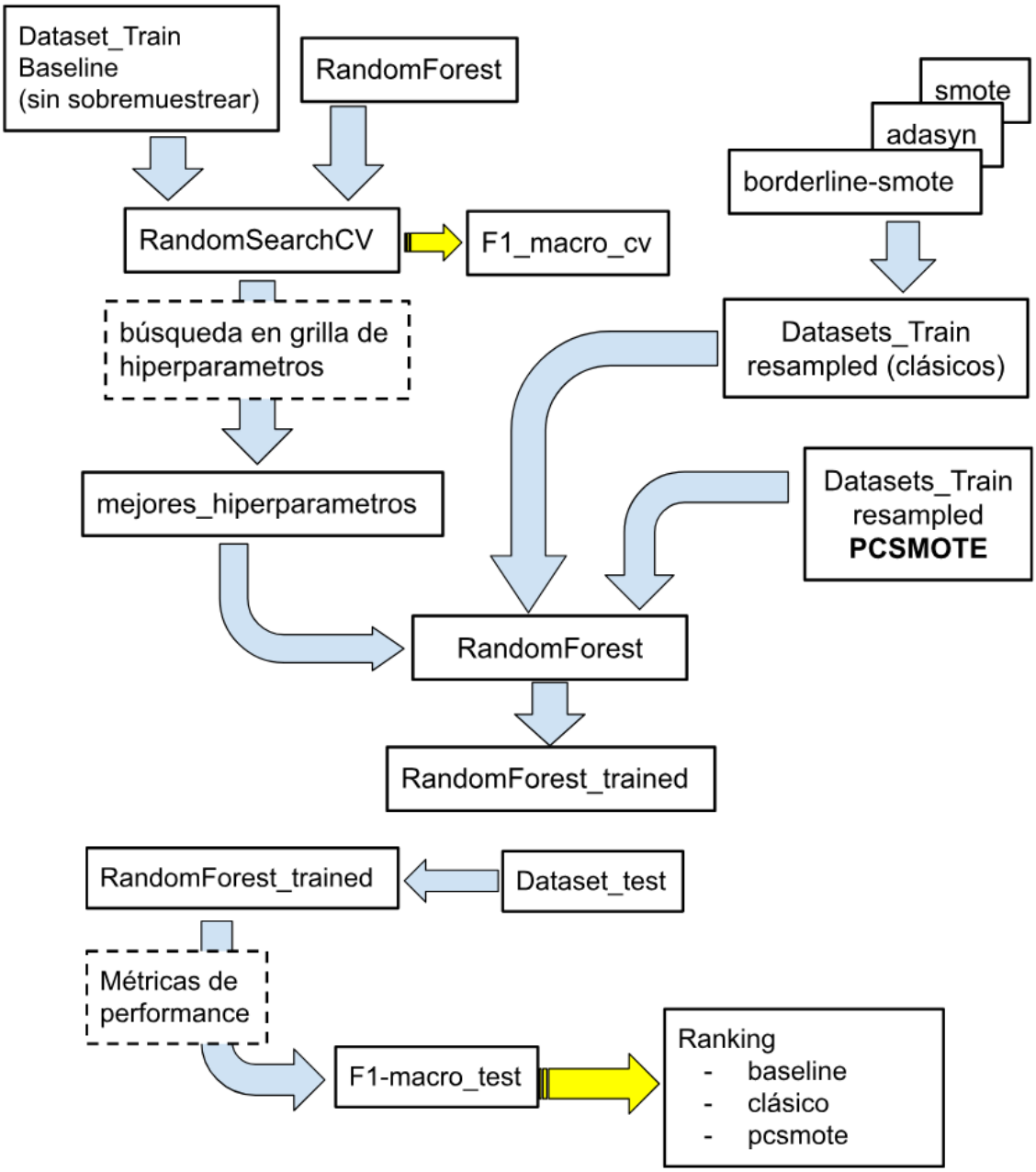

Figura 3.5: Etapa de Train y Test del Modelo de Aprendizaje

4. RESULTADOS

En este capítulo se presentan los hallazgos empíricos derivados de la evaluación de *PCSMOTE* sobre los conjuntos de datos seleccionados. El análisis se estructura partiendo de la validación de los mecanismos de auto-calibración (Fase 1 del algoritmo) para luego detallar el impacto en el rendimiento del clasificador.

Los resultados demostraron que la optimización de estos hiperparámetros topológicos posibilitaron al algoritmo adaptar dinámicamente su comportamiento a los conjuntos de datos analizados. Como así también se identificó un aporte adicional de utilidad en cuanto a la geometría espacial del dataset; es decir, el hecho de que *PCSMOTE* no genere sintéticos o los genere a partir de pocas semillas candidatas brindó información de interés. Se constató que, en algunos casos, los datasets analizados poseían la compatibilidad geométrica necesaria para métodos basados en vecindad y, en otros, presentaron un solapamiento en la estructura global, lo que hizo ineficiente la interpolación local.

4.1. Análisis de la auto-calibración Geométrica

Los resultados expuestos en la **Tabla 4.1** muestran las mejores configuraciones encontradas de *PCSMOTE* con cada dataset. Es dable destacar que estos valores fueron escalados, ya que de lo contrario la técnica se volvería imprecisa al trabajar con magnitudes de distancia diferentes.

La tabla muestra cómo *PCSMOTE* pudo adaptar los hiperparámetros de radio-densidad, radio-riesgo y entropía a la geometría del dataset gracias al uso de los percentiles. Esto se puede observar entre los datasets *Predict Faults* y *Us Crime*, donde se nota cómo la definición de "cerca" varía entre un conjunto de datos y otro. En *Predict Faults*, las muestras están tan juntas que incluso un percentil alto sigue tomando un valor de radio físico pequeño, lo que indica una alta concentración de los datos.

Por el contrario, en *Us Crime* y *Shuttle* se observó un valor de radio físico más grande; es decir, se expandió drásticamente para encontrar vecinos en un espacio disperso de alta dimensión. Al cambiar la noción de "cerca" en estos conjuntos, se observó una mejora en la performance del clasificador y un mejor aporte por parte de *PCSMOTE*.

Tabla 4.1: Auto-calibración de percentiles por dataset y parámetros

Datasets / Hiperparámetros	**Shuttle**	**Telco Churn**	**Predict Faults**	**US Crime**
PR_{dens} (Radio densidad)	70 (medio)	85 (alto)	85 (alto)	85 (alto)
Radio físico densidad	0,40 $\sim$ 981*	1,43	0,582	18,2
PR_{riesgo} (Radio riesgo)	30 (muy bajo)	40 (bajo)	50 (medio)	40 (bajo)
Radio físico riesgo	0,2 $\sim$ 503*	1	0,319	6,8
$PR_{\text{entropía}}$	–	70	–	80
Valor entropía	–	0,59	–	0,86
Criterio pureza	proporción	entropía	proporción	entropía
Densidad (umbral densidad)	25	60	45	45
Riesgo (umbral riesgo)	30	45	55	45
Pureza (umbral prop.)	40	–	45	–
Grado de Isolation Forest	1	0	0	0

Nota. * Rango por clase.

4.2. Análisis de Radios

Se hizo necesario este apartado sobre el análisis de radios, ya que *PCSMOTE* trabajó sobre esta estructura para la primera fase de calibración de los valores de radio-densidad y riesgo. Para ello, se puede observar la primera figura donde la curva muestra una distribución acumulada de las distancias euclídeas entre las instancias minoritarias y sus k vecinos más cercanos (ver Figura 4.10).

El gráfico valida la necesidad de la auto-calibración, ya que dentro del mismo dataset (ver Figura 4.1 y Figura 4.2 - *Shuttle*), la Clase 1 requiere un radio de apenas 0,33 para alcanzar el percentil 70, mientras que para la Clase 6 necesita un radio de 981,5 para el mismo percentil, lo que demuestra que un radio fijo habría fallado en capturar ambas estructuras simultáneamente, mientras que *PCSMOTE* adaptó dinámicamente la escala espacial para cada clase.

En el caso de *Telco Churn*, la curva de radio destaca por su forma creciente con aumentos repentinos. Este fenómeno indica que la clase minoritaria no presenta una densidad binaria (denso/disperso), sino una estratificación estructural en cuatro bloques de densidad decreciente: el **primer segmento (rango percentil 0-20 %)**, caracterizado por radios mínimos (radio $< 0{,}5$), corresponde a los centroides de los *clusters* donde la redundancia de información es máxima; el **segundo bloque (rango percentil 20-55 %)** evidencia un incremento escalonado del radio, indicando la presencia de sub-grupos o *small disjuncts* que requieren una mayor tolerancia espacial para ser conectados; el **tercer tramo (rango percentil 55-85 %)** representa la periferia de las estructuras, donde la densidad disminuye pero mantiene la conectividad suficiente para validar la interpolación (entre 3/7 a 4/7 vecinos dentro del umbral) y, finalmente, el **último segmento** exhibe un crecimiento vertical asintótico, correspondiente a *outliers* globales y ruido desconectado de cualquier estructura latente.

Este análisis confirma que la elección del Percentil de Densidad al 85 % no fue arbitraria, sino geométricamente óptima. Este corte permitió al algoritmo contener los tres primeros bloques estructurales (información útil) y excluir el cuarto bloque de dispersión, maximizando la generación sin introducir ruido sintético.

La correlación entre la morfología gráfica y los datos operativos se confirma mediante la auditoría de los *logs* generados por *PCSMOTE*. Específicamente, se constata que el cuarto bloque (Nube de Dispersión) corresponde materialmente a las semillas que presentaron una densidad vecinal crítica, registrando densidades del orden de 0/7, 1/7 y 2/7 vecinos válidos. Esta baja conectividad física valida la decisión del algoritmo de clasificar estas instancias como ruido u *outliers*, excluyéndolas del proceso de generación para preservar la integridad de la frontera de decisión.

Traducción Operativa del Análisis Geométrico (Telco Churn): La estratificación topológica observada en este conjunto de datos, particularmente el descarte del último bloque asintótico (nube de dispersión) evaluado mediante la entropía de Shannon, posee un impacto directo en el dominio del negocio de las telecomunicaciones. Desde una perspectiva operativa, los vecindarios densos y seguros donde PC-SMOTE permitió generar datos (rango percentil 0-55 %) representan perfiles demográficos claros y predecibles de usuarios propensos a abandonar el servicio (churn); es decir, el núcleo del problema comercial.

Por el contrario, las muestras ubicadas en la nube de dispersión, donde la entropía alcanza su máximo nivel de caos (mezcla 50/50 de clases), representan eventos completamente atípicos (por ejemplo, un cliente leal que cancela abruptamente por una mudanza imprevista). Forzar al algoritmo a generar datos en este espacio ruidoso le haría creer a la

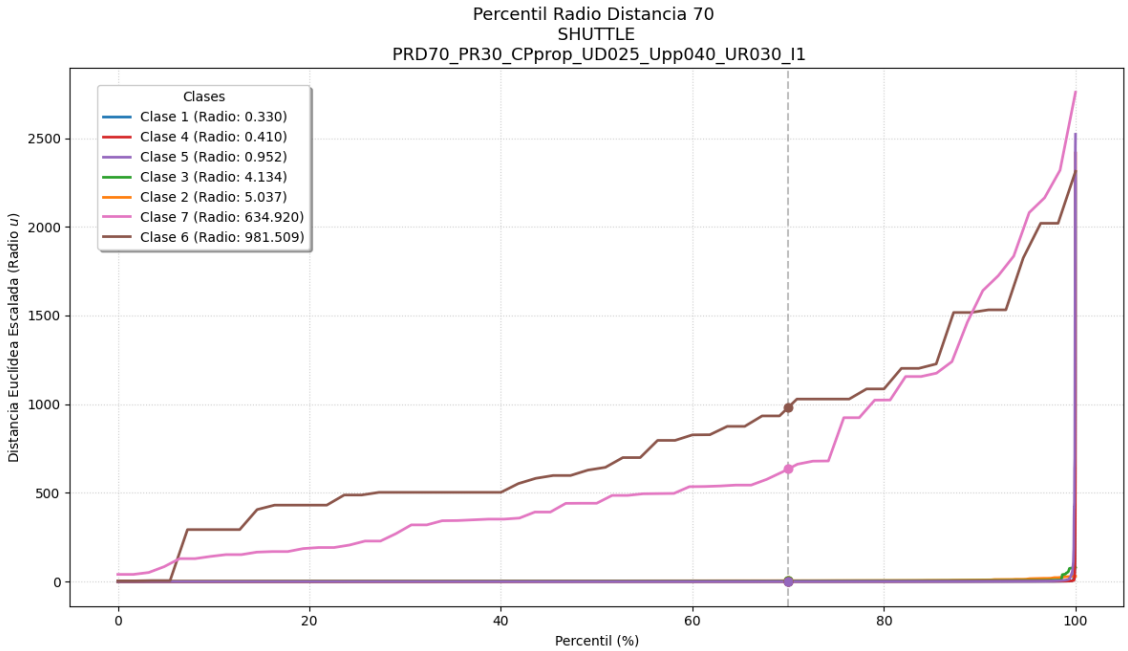

Figura 4.1: Visualizacion de radios por clase del dataset Shuttle.

empresa que existe un patrón de abandono donde solo hay azar, llevando a la organización a desperdiciar cuantiosos recursos financieros en campañas de marketing inútiles para retener a usuarios cuya decisión obedece a factores externos impredecibles. Al detectar esta alta entropía y detenerse, PC-SMOTE ahorra dichos costos operativos.

4.3. Dinámica de Filtrado y Comportamiento adaptativo de PCSMOTE

El análisis transversal de los experimentos nos muestra cómo *PCSMOTE* no operó con una tasa de generación uniforme, sino que actuó como un filtro selectivo dependiente de la morfología de los datos. La **Figura 4.6** muestra dos comportamientos opuestos en la selección de semillas candidatas sobre el estudio de pureza de un dataset: frente a escenarios muy mezclados, el algoritmo optó por no usar la mayoría de las semillas (ver **Figura 4.9** - caso *predict_faults*); por el contrario, cuando la topología del vecindario fue apta para la generación de semillas candidatas, el algoritmo aumentó significativamente el porcentaje de aceptación de las mismas (ver **Figuras 4.7** y **4.8** - *telco churn* y *shuttle*).

Esto confirmó que la técnica, logró identificar escenarios con buena separabilidad local, permitiendo la interpolación intensiva sin comprometer la frontera.

En los gráficos radar, la arista "Reparto de Semillas" nos dice si la generación de semillas está concentrada en muchas o pocas muestras; cuanto más se acerca a 1 más repartido está, al contrario cuanto más se acerca a cero, la generación se concentra en pocas candidatas.

A pesar de la alta cobertura de clases observada en algunos casos (como en el caso Shuttle, Figura 4.10), se concluye que esto no es suficiente. Es indispensable que la generación de semillas sintéticas se distribuya en una mayor cantidad de semillas para prevenir el sobreajuste (overfitting).

Es interesante ver cómo las figuras 4.12 y 4.13 de radar en Us Crime y Predict Faults son muy similares sin embargo pcsmote se comporto diferente en ambos casos. En el primer caso con Us Crime se observó una interpolación que el clasificador pudo aprovechar y

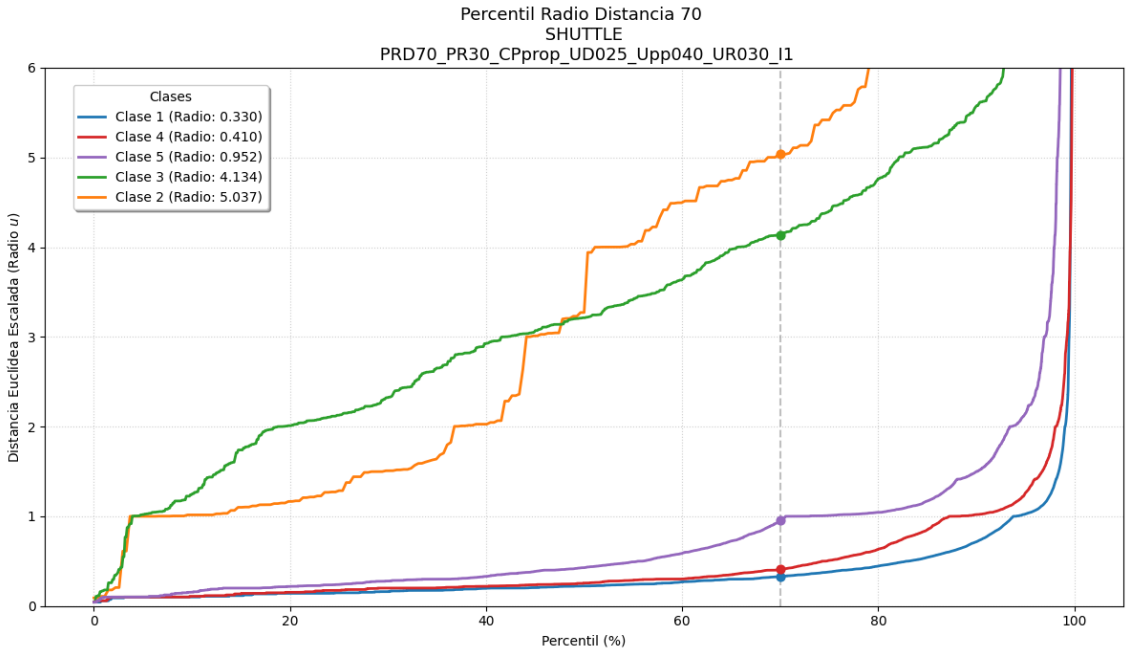

Figura 4.2: Visualizacion de radios con foco en las clases 1 a 5 del dataset Shuttle.

mejorar el poder de generación, y en el segundo caso a pesar de identificar la estructura la intervención de la técnica genero una deficiente performance del clasificador no pudiendo superar el caso base (ver Tablas 4.13 y 4.4 Us Crime y Predict Faults).

Asimismo, también se pudo observar una relación entre "semillas sintéticas válidas" y "reparto de semillas", es decir que, si hay muchas semillas que pasaron los filtros, luego repercute en una generación más concentrada de sintéticos. Al contrario, cuando pocas semillas pasaron los filtros, la generación de sintéticos es más distribuida.

El mismo fenomeno ocurrio en el caso del dataset Us Crime, donde se alcanzó a cubrir todas las clase en la generación de sintéticos y también la distribución de sintéticos estuvo muy bien repartida en casi la totalidad de las semillas que superaron todos los filtros, sin embargo en esta ocasión, PCSMOTE filtró demasiadas semillas ya que el dataset era muy ruidoso y bastante mezclado.

Traducción Operativa del Análisis Geométrico (Predict Faults): El comportamiento de PC-SMOTE en el dataset *Predict Faults* (fallas industriales), donde el algoritmo actuó como un estricto sensor de inviabilidad rechazando casi la totalidad de las semillas candidatas, revela una lectura física de necesario analisis. En este sentido, la alta mezcla topológica detectada significa operativamente que las maquinarias que están a punto de fallar operan bajo parámetros físicos y térmicos casi indistinguibles de las máquinas que funcionan en perfecto estado.

En este escenario, el borde de decisión es imperceptible, ya que si se hubiese utilizado un enfoque tradicional de sobremuestreo a ciegas, el algoritmo habría inventado perfiles de fallas irreales que no existen verdaderamente en la planta industrial, llevando a los ingenieros a detener líneas de producción basados en falsas alarmas, generando posibles pérdidas en el mundo real. Por lo que se puede afirmar que el acierto de la técnica aqui presentada no radica en maximizar ciegamente una metrica de performance como F1, sino en su capacidad de negarse a generar datos ruidosos, evitando de forma proactiva falsos positivos con consecuencias industriales no deseables.

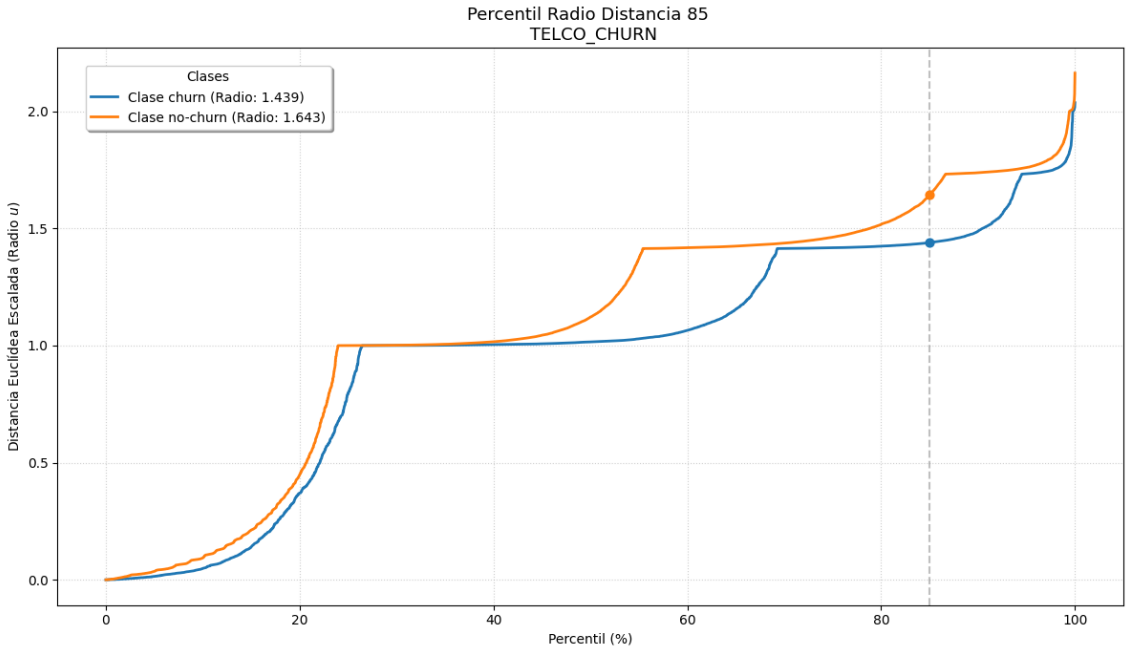

Figura 4.3: Visualizacion de radios por clase del dataset Telco Churn.

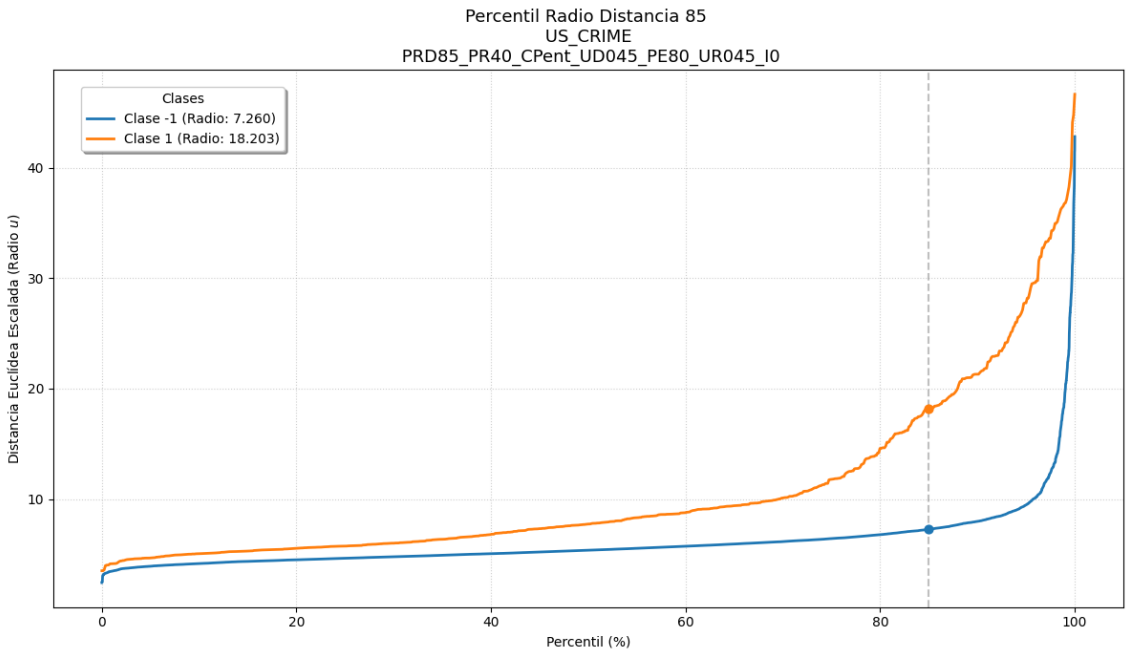

Figura 4.4: Visualizacion de radios por clase del dataset US Crime.

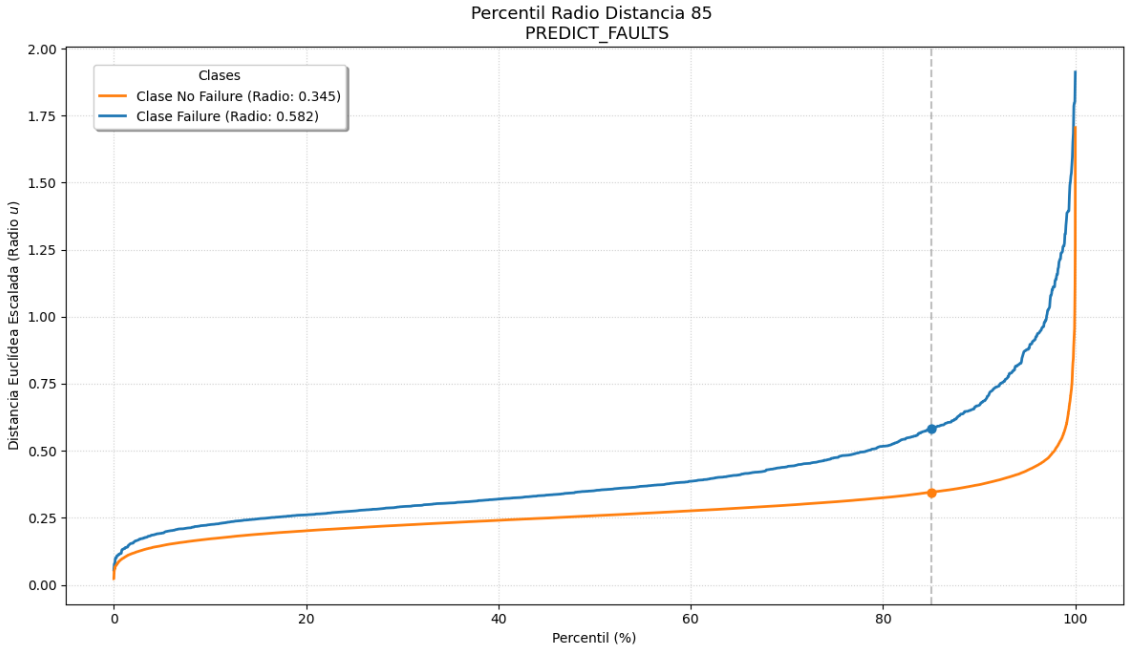

Figura 4.5: Visualizacion de radios por clase del dataset Predict Faults.

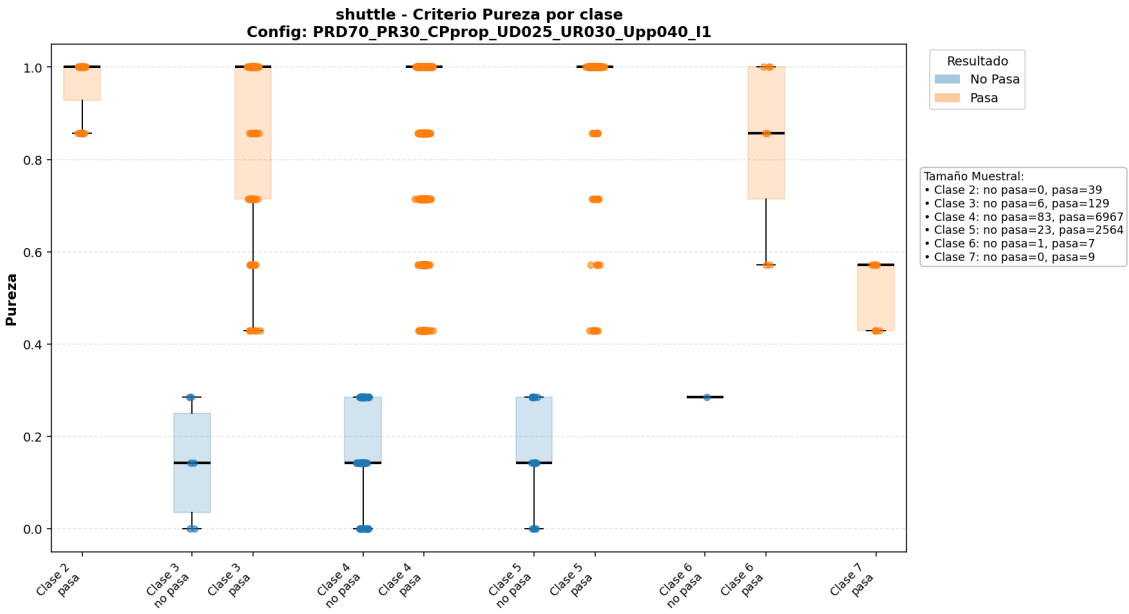

Figura 4.6: Visualizacion de purezas por clase del dataset Shuttle.

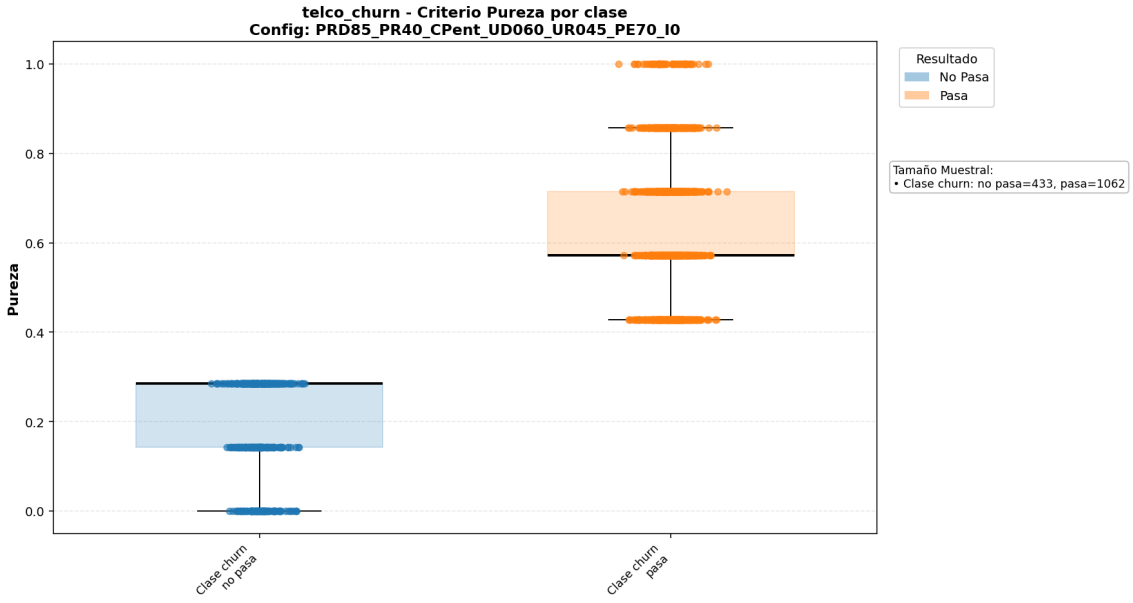

Figura 4.7: Visualizacion de purezas por clase del dataset Telco Churn.

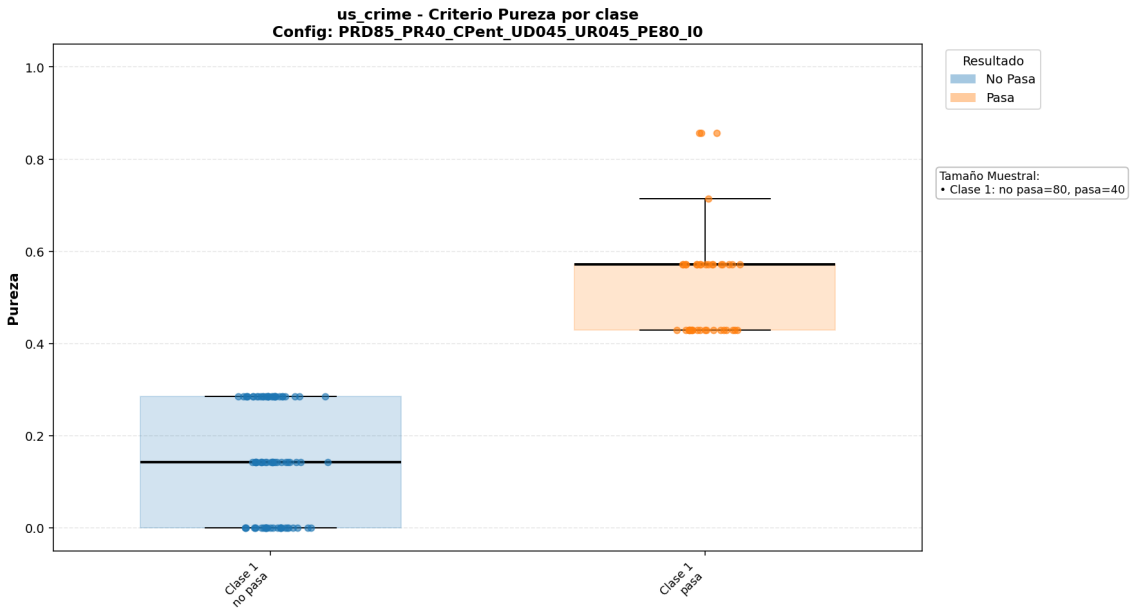

Figura 4.8: Visualizacion de purezas por clase del dataset US Crime.

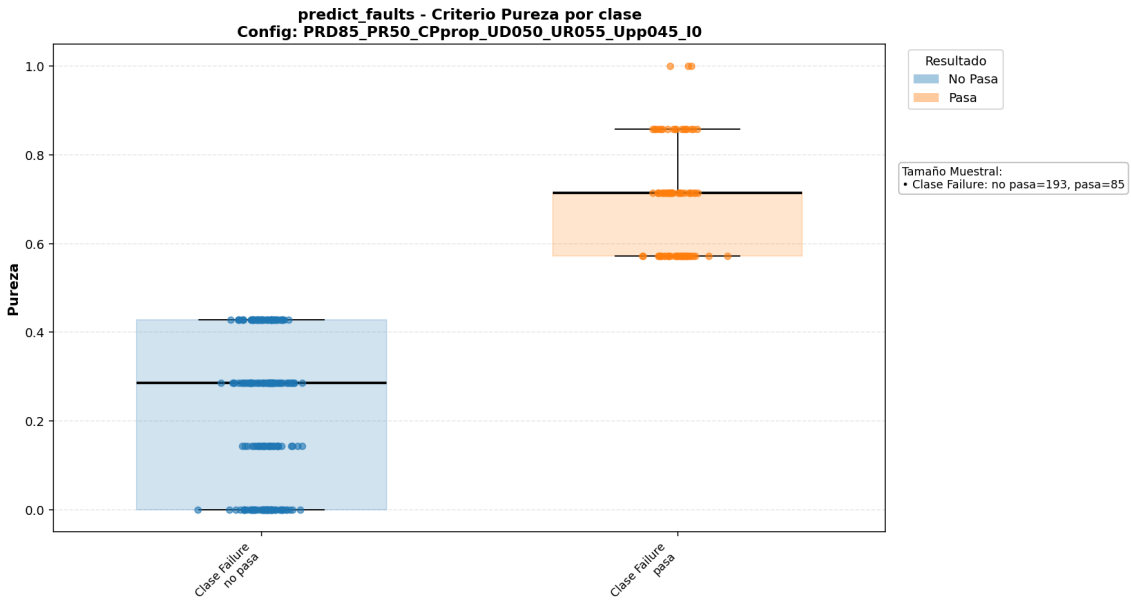

Figura 4.9: Visualizacion de purezas por clase del dataset Predict Faults.

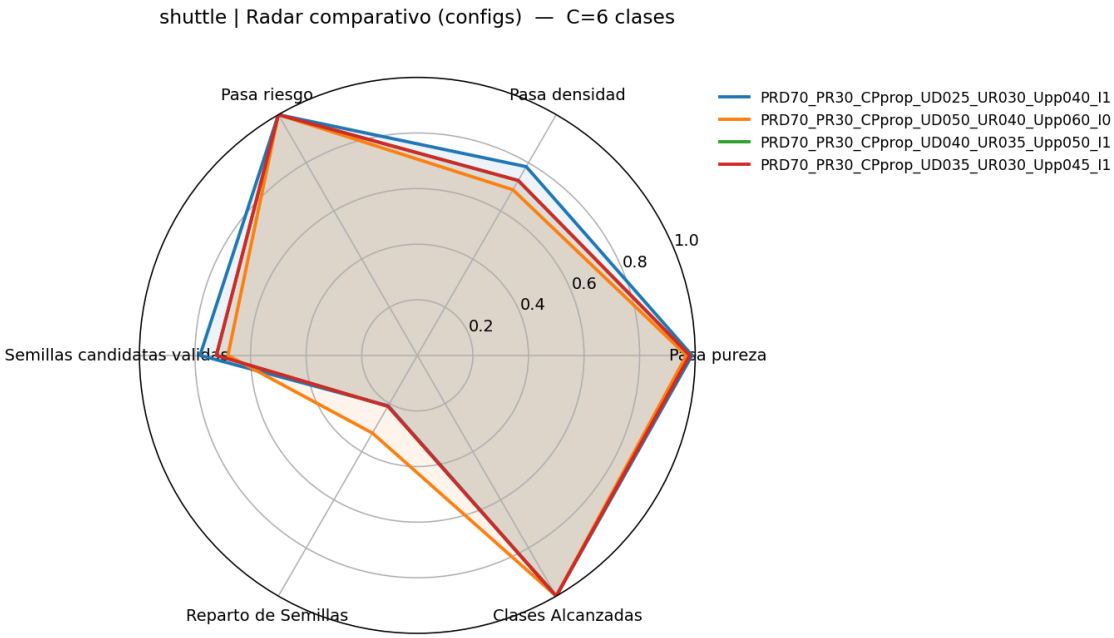

Figura 4.10: Radar comparativo (configs) del dataset Shuttle.

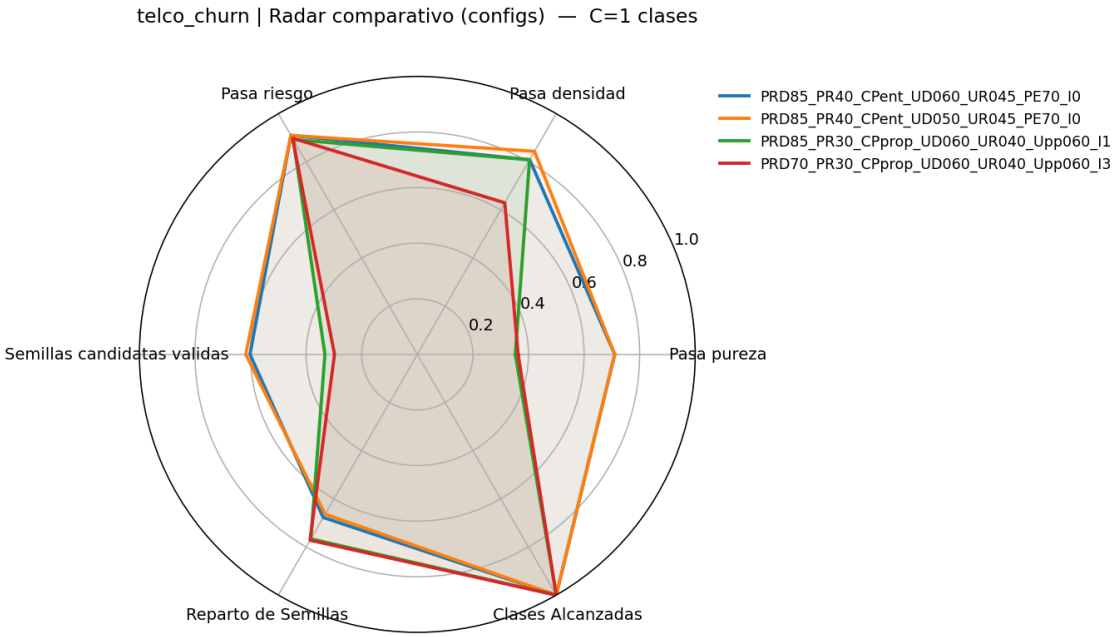

Figura 4.11: Radar comparativo (configs) del dataset Telco Churn.

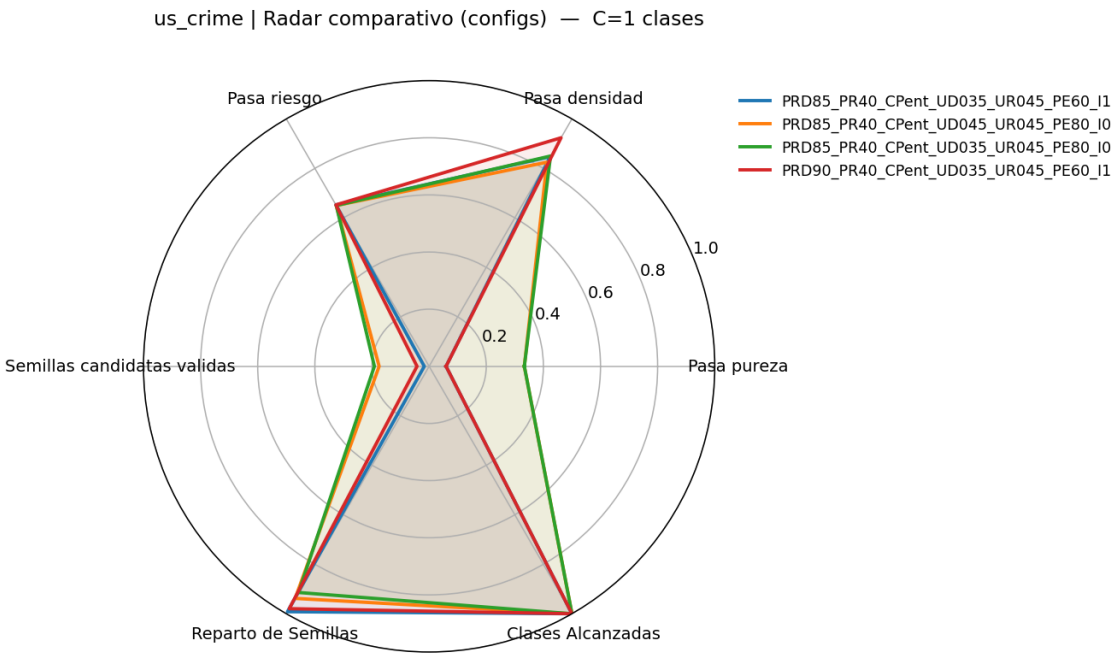

Figura 4.12: Radar comparativo (configs) del dataset US Crime.

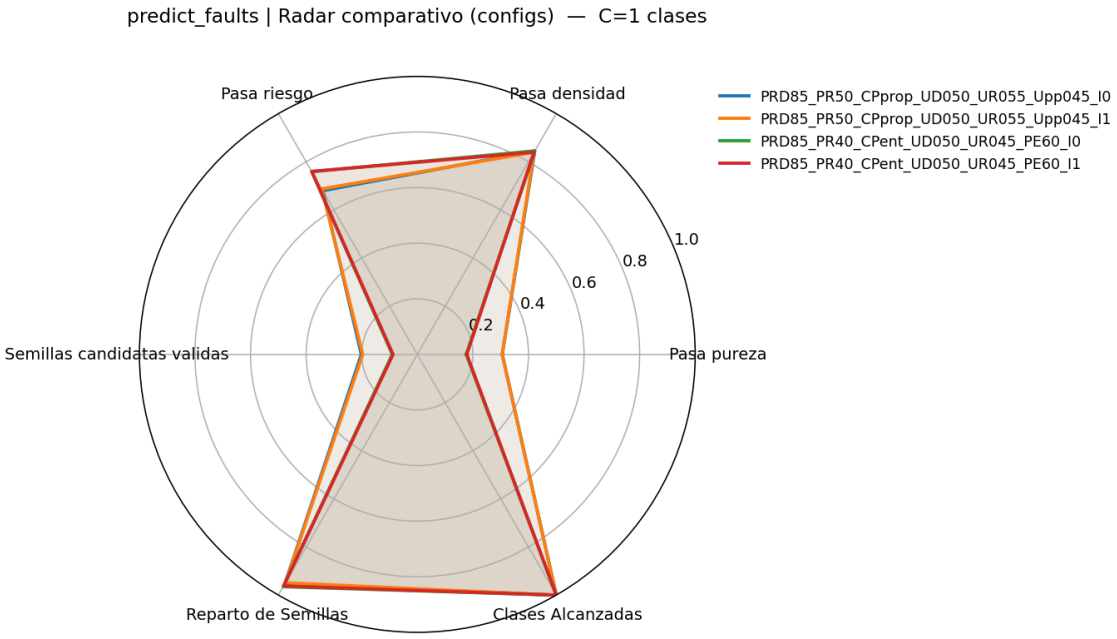

Figura 4.13: Radar comparativo (configs) del dataset Predict Faults.

4.4. Análisis de resultados de la implementación de PCSMOTE

Para un análisis más detallado y descriptivo, los resultados se presentan en tres tablas distintas para cada conjunto de datos analizado. La **Primera Tabla** muestra las configuraciones de *PCSMOTE* evaluadas, ordenadas de acuerdo con la mejor puntuación obtenida en la métrica F1-macro. La **Segunda Tabla** presenta el desempeño logrado en el caso base (*baseline*), incluyendo los tamaños de los conjuntos de entrenamiento (*train*) y prueba (*test*), junto con los valores obtenidos de las métricas. Finalmente, la **Tercera Tabla** contiene la comparación de cada experimento realizado ($E1, E2$, etc.) con el caso base, lo que permite visualizar claramente la eficiencia de la técnica aplicada.

La tabla comparativa se interpreta de la siguiente manera: la primera columna, "Percentil de Isolation Forest", indica el percentil de dicha técnica aplicado para la limpieza de *outliers* (un valor de 0 % significa que no se aplicó esta limpieza). Entre paréntesis, se detallan las "Semillas Candidatas", que representan la cantidad de semillas analizadas. Luego, se especifica la "Técnica y Semillas Válidas", mostrando el método de sobremuestreo aplicado y la cantidad de semillas que resultaron aptas. En la columna "Semillas Válidas (Sem. Val.)", se reporta la cantidad de muestras sintéticas generadas. Se aclara que, en algunos casos, *PCSMOTE* no produce la misma cantidad que el *SMOTE* clásico; si ninguna muestra de una clase supera los filtros, no se generan sintéticos para ella. No obstante, si al menos una semilla resulta válida, la generación se realiza igualando el número de muestras de la clase mayoritaria.

La columna subsiguiente, denominada "Muestras Totales (Train + Sintéticos)", indica la suma de las muestras originales de entrenamiento y las sintéticas generadas. Finalmente, se incluyen las "Métricas de Performance", que consisten en tres indicadores clave para facilitar un análisis profundo y la comparación con otros trabajos. Entre estas métricas destaca la *F1-macro*, calculada como el promedio aritmético de los puntajes F1 obtenidos por cada clase para evaluar la capacidad del clasificador *Random Forest*. Esta métrica revela la eficacia con la que el modelo identifica correctamente las muestras frente a la cantidad de errores cometidos.

Este desempeño depende en gran medida de la calidad informativa de los datos de entrenamiento; si una técnica de sobremuestreo genera muestras sintéticas idénticas, resultaría en información redundante (*sobreajuste*), lo cual obstaculiza el trabajo del clasificador.

De lo anteriormente dicho, puede observarse que, en aquellos casos donde *PCSMOTE* pudo cubrir la generación de sintéticos en todas las clases, el clasificador obtuvo un alto desempeño, como en el caso de *Shuttle* (**Figura 4.10**) e inclusive cuando el dataset era más hostil (vecindades ruidosas y mezcladas) como en *Us Crime* (**Figura 4.12**).

Al analizar los resultados del dataset *Predict Faults* (Tabla 4.4 y Figura 4.13), se observa el principal triunfo arquitectónico de la técnica: la ejecución de una **negación táctica** de datos. Frente a un escenario de extremo solapamiento industrial, donde las maquinarias a punto de fallar operan con parámetros físicos casi idénticos a las sanas, PCSMOTE detectó la inviabilidad del espacio y rechazó deliberadamente la interpolación masiva de semillas.

Si bien el *baseline* y las técnicas clásicas (como SMOTE y ADASYN) pueden forzar la generación de datos en esta zona de caos para inflar artificialmente la métrica F1-macro, operativamente esto representa un error crítico. Obligar a un algoritmo a interpolar a ciegas en un solapamiento masivo hace que el modelo alucine patrones de riesgo espurios, creando verdaderas "máquinas fantasma"que no existen en la realidad. En un entorno de producción, este sobreajuste desencadenaría falsas alarmas y la detención innecesaria de líneas de producción, generando pérdidas millonarias.

Por lo tanto, que PC-SMOTE presente una restricción en la generación de muestras y no busque superar ciegamente el F1-macro del caso base no es una deficiencia, sino la demostración empírica de su éxito. El algoritmo actuó como un **escudo protector topológico**, priorizando la integridad absoluta de la frontera de decisión y la seguridad operativa de la industria por encima de la optimización de una métrica teórica engañosa.

Es por ello que, lejos de representar una falla operativa del algoritmo, esta retracción constituye una respuesta topológica adaptativa ante un entorno de extrema complejidad. En escenarios de solapamiento crítico, la frontera de decisión que separa a las clases se vuelve sumamente difusa y frágil. Si el algoritmo hubiera operado a ciegas —como lo hacen las técnicas tradicionales— inyectando cientos de datos sintéticos justo en el medio de esta región altamente ruidosa, habría destruido cualquier límite sutil existente, garantizando un sobreajuste masivo que colapsaría irremediablemente el rendimiento del modelo en producción.

En este sentido, la limitación en la generación de muestras evidencia que PC-SMOTE es capaz de identificar la inviabilidad del espacio, priorizando la preservación y la integridad de la frontera de decisión por encima del simple balanceo numérico de las clases. El algoritmo actúa, por lo tanto, como un mecanismo de seguridad estructural.

Tabla 4.2: Configuración de hiperparámetros por técnica y Umbral de limpieza (Predict Faults)

Exp.	Umbral IF*	Técnica	PR_dist	PR_ries	Criterio	U_den	U_ries	Tipo_P
E1	Umbral 0: no se filtró por IF (candidatas: 278)	**pcsmote**	**85**	**40**	**entropía**	**050**	**045**	**PE60**
E2		pcsmote	85	50	proporción	050	055	Upp045
E3	P 1 % (275)	**pcsmote**	**85**	**40**	**entropía**	**050**	**045**	**PE60**
E4		pcsmote	85	50	proporción	050	055	Upp045
E5	P 3 % (269)	**pcsmote**	**85**	**50**	**proporción**	**050**	**055**	**Upp045**

* El valor entre paréntesis indica las semillas sintéticas candidatas.

Tabla 4.3: Rendimiento en Validación Cruzada (k-fold CV) para el Caso Base (Predict Faults)

Configuración	Tamaño Train (80 %)	Tamaño Test (20 %)	F1_M CV	F1_W CV	Acc CV
Caso Base (Original)	8000	2000	0,865	0,983	0,985

Nota. El conjunto de entrenamiento (80 %) presenta un fuerte desbalance de clases: 7722 instancias pertenecen a la clase mayoritaria (-1) y solo 278 a la clase minoritaria (1).

Tabla 4.4: Rendimiento final en conjunto de prueba: Caso Base vs. Sobremuestreo (Predict Faults)

Exp.	Umbral IF	Técnica	Semillas Validadas*	Sintet.	Train Final (Orig + Sintet.)	F1_M Test	Δ (F1_M)	F1_W Test	Acc Test
–	**Caso Base**	–	–	–	8000	0,856	–	0,982	0,983
E1	Umbral 0: no se filtro por IF (candidatas: 278)	**pcsmote**	**25**	**7444**	**15444**	**0.828**	**-0.028**	**0.978**	**0.980**
E2		pcsmote	56	7444	15444	0,822	−0,034	0,978	0,980
–		borderline	–	7444	15444	0,825	−0,031	0,976	0,977
–		adasyn	–	7453	15453	0,813	−0,043	0,974	0,973
–		smote	–	7444	15444	0,808	−0,048	0,973	0,973
E3	P 1 % (candidatas: 275)	**pcsmote**	**24**	**7369**	**15288**	**0.839**	**-0.017**	**0.979**	**0.981**
E4		pcsmote	54	7369	15288	0,828	−0,028	0,978	0,980
–		borderline	–	7369	15288	0,827	−0,029	0,976	0,976
–		smote	–	7369	15288	0,829	−0,027	0,976	0,975
–		adasyn	–	7380	15299	0,815	−0,041	0,974	0,973
E5	P 3 % (candidatas: 269)	**pcsmote**	**53**	**7221**	**14980**	**0.839**	**-0.017**	**0.979**	**0.980**

* Semillas candidatas validadas en el conjunto de entrenamiento (*train*). Δ ($F1_M$) representa la diferencia absoluta respecto al Caso Base. *Nota:* Las filas en negrita resaltan los experimentos con mejor desempeño relativo.

Tabla 4.5: Configuración de hiperparámetros para experimentos seleccionados (Shuttle)

Exp.	Umbral IF*	Técnica	PR_dist	PR_ries	Criterio	U_den	U_ries	Tipo_P
E1	1 (9828)	*pcsmote*	70	30	proporción	025	030	Upp040
E2	0 (9931)	*pcsmote*	70	30	proporción	050	040	Upp060

* Valor entre paréntesis indica semillas sintéticas candidatas detectadas.

Tabla 4.6: Rendimiento en Validación Cruzada (k-fold CV) para el Caso Base (Shuttle)

Configuración	Tamaño Train	Tamaño Test	F1_M CV	F1_W CV	Acc CV
Caso Base (Original)	46400	11600	0,963	1,000	1,000

Nota. El conjunto de entrenamiento (80 %) se compone de 36469 instancias de la Clase 1 (mayoritaria), y las clases minoritarias: Clase 4 (7122), Clase 5 (2614), Clase 3 (137), Clase 2 (40), Clase 7 (10) y Clase 6 (8).

Este mecanismo tambien se pudo observar en el conjunto de datos shuttle (Tabla 4.7), mas precisamente en la generación de sintéticos por parte del algoritmo donde solo se crearon 135.963 muestras, a diferencia de los 208.883 generados por el SMOTE clásico. Esta sutil pero llamativa reducción numerica es una característica del funcionamiento de pcsmote, y se manifestó de forma más marcada en ambas aplicaciones en conjunto con Isolation Forest.

Específicamente, con Isolation Forest Umbral Percentil 0, pcsmote no pudo identificar semillas candidatas para las clases minoritarias 6 y 7, lo que lo llevó a centrarse en nivelar las clases 2 a 5 contra la mayoritaria (clase 1). No obstante, la aplicación de una limpieza previa de outliers con un umbral del 1 % resultó en una mejora del rendimiento.

Esta mejora se atribuye a dos factores clave: la variación en el umbral del radio de densidad y riesgo, y la relajación del umbral de densidad al 0.25 (2/7) en la cantidad de vecinos cercanos. Esto permitió que las clases 6 y 7 fueran consideradas suficientemente

densas al encontrar vecinos, logrando su inclusión en el conjunto y, consecuentemente, un aumento en la performance general.

Tabla 4.7: Rendimiento final en conjunto de prueba ordenado por F1 Macro (Shuttle)

Ranking	Umbral IF	Técnica	Sem. Val.*	Sintet.	Train Final (Orig + Sintet.)	F1_M Test	Δ (F1_M)	F1_W Test	Acc Test
–	–	**Caso Base**	–	–	46400	0,855	–	1,000	1,000
1	Umbral 0: no se filtró por IF (candidatas: 9931)	*adasyn*	–	208961	255361	0,952	+0,097	1,000	1,000
2		*borderline*	–	172454	218854	0,952	+0,097	1,000	1,000
3		*smote*	–	208883	255283	0,950	+0,095	1,000	1,000
4		**pcsmote (E2)**	**6756**	**135963**	**182363**	**0.857**	**+0.002**	**1.000**	**1.000**
1	P 1 % (candidatas: 9828)	*adasyn*	–	206831	252763	0,991	+0,136	1,000	1,000
2		*borderline*	–	170731	216663	0,991	+0,136	1,000	1,000
3		*smote*	–	206796	252728	0,950	+0,095	1,000	1,000
4		**pcsmote (E1)**	**7685**	**206796**	**252728**	**0.950**	**+0.095**	**1.000**	**1.000**

* Semillas candidatas validadas en el conjunto de entrenamiento (*train*). Δ (F1_M) representa la mejora absoluta respecto al Caso Base. *Nota:* Las técnicas están ordenadas por F1_M Test dentro de cada bloque.

Tabla 4.8: Configuración de hiperparámetros para experimentos seleccionados (Telco Churn)

Exp.	Umbral IF*	Técnica	PR_dist	PR_ries	Criterio	U_den	U_ries	Tipo_P
E1	1 (1480)	*pcsmote*	85	30	proporción	060	040	Upp060
E2	0 (1495)	*pcsmote*	85	40	entropía	060	045	PE70

* Valor entre paréntesis indica semillas sintéticas candidatas detectadas.

Tabla 4.9: Rendimiento en Validación Cruzada (k-fold CV) para el Caso Base (Telco Churn)

Configuración	Tamaño Train	Tamaño Test	F1_M CV	F1_W CV	Acc CV
Caso Base (Original)	5634	1409	0,711	0,782	0,790

Nota. El conjunto de entrenamiento (80 %) presenta un desbalance de clases: 4139 instancias de la clase mayoritaria (-1) y 1495 de la clase minoritaria (1).

Tabla 4.10: Rendimiento final en conjunto de prueba ordenado por F1 Macro (Telco Churn)

Ranking	Umbral IF	Técnica	Sem. Val.*	Sintet.	Train Final (Orig + Sintet.)	F1_M Test	Δ (F1_M)	F1_W Test	Acc Test
–	0 %	**Caso Base**	–	–	5634	0,705	–	0,779	0,789
–	1 %	**Caso Base**	–	–	5577	0,702	–	0,776	0,786
1	0 % (candidatas: 1495)	*smote*	–	2644	8278	0,711	+0,006	0,782	0,790
2		*adasyn*	–	2594	8228	0,710	+0,005	0,773	0,771
3		**pcsmote (E2)**	**897**	**2644**	**8278**	**0.708**	**+0.003**	**0.775**	**0.778**
4		*borderline*	–	2644	8278	0,708	+0,003	0,770	0,768
1	1 % (candidatas: 1480)	**pcsmote (E1)**	**495**	**2617**	**8194**	**0.715**	**+0.013**	**0.783**	**0.788**
2		*smote*	–	2617	8194	0,713	+0,011	0,776	0,776
3		*adasyn*	–	2535	8112	0,712	+0,010	0,776	0,771
4		*borderline*	–	2617	8194	0,703	+0,001	0,766	0,763

* Semillas candidatas validadas en *train*. Δ ($F1_M$) representa la mejora absoluta respecto al Caso Base. *Nota:* Las técnicas están ordenadas por $F1_M$ Test para mostrar el ranking real de *pcsmote*.

Tabla 4.11: Configuración de hiperparámetros para experimentos seleccionados (US Crime)

Exp.	Umbral IF*	Técnica	PR_dist	PR_ries	Criterio	U_den	U_ries	Tipo_P
E1	0 (120)	*pcsmote*	85	40	entropía	0,45	0,45	PE80
E2	1 (118)	*pcsmote*	85	40	entropía	0,35	0,45	PE60
E3	3 (116)	*pcsmote*	90	45	proporción	0,35	0,50	Upp060

* Valor entre paréntesis indica semillas sintéticas candidatas detectadas.

Tabla 4.12: Rendimiento en Validación Cruzada (k-fold CV) para el Caso Base (US Crime)

Configuración	Tamaño Train	Tamaño Test	F1_M CV	F1_W CV	Acc CV
Caso Base (Original)	1595	399	0,713	0,930	0,941

Nota. El conjunto de entrenamiento (80 %) presenta un fuerte desbalance de clases: 1475 instancias pertenecen a la clase mayoritaria (-1) y solo 120 a la clase minoritaria (1).

Tabla 4.13: Rendimiento final: Comparativa de *pcsmote* vs. Técnicas Clásicas (US Crime)

Exp.	Umbral IF	Técnica	Sem. Val.*	Sintet.	Train Final (Orig + Sintet.)	F1_M Test	Δ (F1_M)	F1_W Test	Acc Test
–	–	**Caso Base**	–	–	1595	0,765	–	0,940	0,945
E1	0 % (candidatas: 120)	**pcsmote**	**21**	**1355**	**2950**	**0.801**	**+0.036**	**0.948**	**0.952**
–		*adasyn*	–	1368	2963	0,752	−0,013	0,928	0,925
–		*smote*	–	1355	2950	0,736	−0,029	0,923	0,920
–		*borderline*	–	1355	2950	0,725	−0,040	0,923	0,922
E2	1 % (candidatas: 118)	**pcsmote**	**2**	**1342**	**2920**	**0.788**	**+0.023**	**0.944**	**0.947**
–		*borderline*	–	1342	2920	0,719	−0,046	0,918	0,915
–		*smote*	–	1342	2920	0,747	−0,018	0,926	0,922
E3	3 % (candidatas: 116)	**pcsmote**	**6**	**1314**	**2860**	**0.742**	**-0.023**	**0.931**	**0.935**
–		*smote*	–	1314	2860	0,715	−0,050	0,916	0,912
–		*adasyn*	–	1304	2850	0,725	−0,040	0,917	0,910
–		*borderline*	–	1314	2860	0,707	−0,058	0,913	0,907

* Semillas candidatas validadas en el conjunto de entrenamiento (*train*). Δ ($F1_M$) representa la ganancia o pérdida absoluta respecto al Caso Base. *Nota:* Los experimentos E1, E2 y E3 resaltan la variante de *pcsmote* con mejor desempeño.

5. CONCLUSIONES Y TRABAJOS FUTUROS

5.1. Conclusiones

En la presente investigación se abordó uno de los desafíos más persistentes en el aprendizaje automático supervisado: la degradación del rendimiento de los clasificadores frente a la sinergia perjudicial entre el desbalance de clases y el solapamiento estructural. Para superar las limitaciones de la interpolación ciega y la sobregeneralización de métodos tradicionales, se desarrolló y validó el algoritmo PC-SMOTE (Percentile-Controlled SMOTE).

La principal contribución de este trabajo radica en la demostración empírica de que la autocalibración geométrica, fundamentada en la extracción de percentiles para definir umbrales de densidad, riesgo y pureza, constituye un mecanismo altamente efectivo para adaptar la escala espacial del sobremuestreo a la topología local de cada clase. Los resultados experimentales, validados mediante un modelo Random Forest sobre conjuntos de datos reales, evidenciaron que PC-SMOTE no solo restringe la selección de semillas y la generación de muestras a regiones topológicamente seguras, sino que logra superar en rendimiento (evaluado a través de la métrica F1-macro) a estrategias clásicas como SMOTE, Borderline-SMOTE y ADASYN en dominios complejos como *Shuttle*, *US Crime* y *Telco Churn*.

5.2. Discusión

Un hallazgo de particular relevancia geométrica durante la etapa experimental fue el comportamiento del algoritmo frente a distintos niveles de solapamiento de clases. Mientras que en datasets con buena separabilidad local (como *US Crime*) PC-SMOTE logró una cobertura y distribución de sintéticos que mejoró sustancialmente el poder discriminatorio del modelo, en escenarios de extrema complejidad y ruido (como *Predict Faults*), la técnica limitó drásticamente la generación de muestras.

Lejos de representar una falla del algoritmo, este comportamiento actuó como un **sensor de inviabilidad topológica**. PC-SMOTE demostró que, frente a un nivel crítico de impureza e invasión de la clase mayoritaria, forzar la interpolación de muestras sintéticas resulta contraproducente y perjudicial para la frontera de decisión. Este cambio de paradigma sugiere que las técnicas de remuestreo no deben generar datos a ciegas para satisfacer un ratio de desbalance, sino que deben supeditar la generación a la viabilidad estructural del vecindario.

5.3. Trabajos Futuros

A partir de las conclusiones obtenidas y considerando los desafíos actuales en la literatura sobre aprendizaje con datos desbalanceados, se proponen las siguientes líneas de investigación para continuar la evolución de PC-SMOTE:

- **Implementación de Métricas de Distancia Alternativas:** La efectividad de la fase de autocalibración de PC-SMOTE depende críticamente del cálculo de vecindades mediante distancias euclidianas. Sin embargo, en espacios de alta dimensionalidad con características irrelevantes, la métrica euclidiana puede perder fiabilidad

debido a la "maldición de la dimensionalidad". Se propone explorar métricas de distancia alternativas, como la distancia de Mahalanobis, que ponderen la relevancia de los atributos y mejoren la identificación de vecinos cercanos [16, 21].

- **Integración con Reducción de Dimensionalidad y Clustering:** Para mitigar el costo computacional en grandes volúmenes de datos (Big Data) y mejorar la separabilidad de las clases antes de la evaluación de densidades y riesgos, se sugiere hibridar PC-SMOTE con técnicas de reducción de dimensionalidad (como PCA) y clustering basado en densidad (como HDBSCAN) [12, 18]. Esto no solo optimizaría la escalabilidad de la técnica, sino que refinaría la detección del ruido sintético.

- **Sistemas de Recomendación mediante Meta-aprendizaje:** Si bien PC-SMOTE optimiza sus radios de forma dinámica, los percentiles iniciales ($PRdens$, $PRriesgo$) deben ser suministrados. Apoyándose en recientes investigaciones, un trabajo futuro prometedor es el desarrollo de un sistema de recomendación que evalúe métricas de complejidad de datos de manera a priori (como la fracción de puntos en la frontera o el solapamiento estructural) para sugerir automáticamente la configuración óptima de los percentiles, eliminando la necesidad de realizar búsquedas en grilla computacionalmente costosas [1].

- **Dinamismo Estructural en el Parámetro de Vecindad (k):** Si bien la versión actual de PC-SMOTE requiere fijar el número de vecinos ($k = 7$) para mantener una unidad de medida estable en el cálculo comparativo de la entropía de Shannon, un desafío matemático de gran relevancia es lograr que este hiperparámetro también abandone su naturaleza estática. Investigaciones futuras podrían explorar la auto-regulación de la variable k basándose en la densidad global y local del conjunto de datos. El objetivo sería desarrollar una escala de entropía normalizada que permita comparar impurezas topológicas sin depender de un denominador constante, logrando así una fluidez espacial y paramétrica absoluta en todas las fases del algoritmo.

6. APÉNDICE: CÓDIGO FUENTE

Este apéndice presenta los fragmentos de código más relevantes de la implementación de PC-SMOTE. Se incluyen los tres filtros de selección de semillas del algoritmo principal, el pipeline de sobremuestreo (tanto con técnicas clásicas como con PC-SMOTE) y el pipeline de entrenamiento y evaluación mediante búsqueda aleatoria de hiperparámetros.

El código fuente completo, junto con los scripts de experimentación y los notebooks de análisis, se encuentra disponible públicamente en el siguiente repositorio de GitHub: `https://github.com/jnatmed/etapa-analisis_pcsmote`.

6.1. Algoritmo PC-SMOTE: Filtros de Selección de Semillas

El núcleo de PC-SMOTE reside en tres filtros geométricos que determinan si una muestra minoritaria es apta para ser utilizada como semilla generadora de muestras sintéticas. Una semilla es considerada **candidata** únicamente si supera los tres filtros simultáneamente:

$$\text{semilla candidata} \iff \text{pureza_ok} \wedge \text{densidad_ok} \wedge \text{riesgo_ok}$$

6.1.1. Fase 1 – Filtro de Pureza

El filtro de pureza evalúa la homogeneidad del vecindario de cada semilla positiva. Se implementan dos criterios alternativos: por **proporción** de vecinos de la misma clase, o por **entropía** del vecindario.

```python
def _calcular_pureza_por_proporcion(self, y_binaria, matriz_indices_vecinos):
    """
    pureza_i = (# vecinos con y==1) / k_vecinos
    """
    y_binaria = np.asarray(y_binaria)
    cantidad_muestras = matriz_indices_vecinos.shape[0]
    purezas = np.zeros(cantidad_muestras, dtype=float)

    for i in range(cantidad_muestras):
        indices_vecinos = matriz_indices_vecinos[i]
        vecinos_misma_clase = sum(
            1 for idx in indices_vecinos if int(y_binaria[idx]) == 1
        )
        purezas[i] = vecinos_misma_clase / float(self.k_vecinos)

    return purezas
```

Listing 6.1: Pureza por proporción de vecinos minoritarios

```python
def _calcular_pureza_por_entropia(self, y_binaria, matriz_indices_vecinos):
    """
    H_i = - sum(p_c * log2(p_c)), c en {0, 1}
    H_i = 0 -> vecindario puro; H_i = 1 -> mezcla maxima (50/50)
```

```
    Filtro: H_i <= (1 - umbral_pureza)
    """
    y_binaria = np.asarray(y_binaria)
    n = matriz_indices_vecinos.shape[0]
    entropias = np.zeros(n, dtype=float)

    for i in range(n):
        p_min = sum(
            1 for idx in matriz_indices_vecinos[i]
            if int(y_binaria[idx]) == 1
        ) / float(self.k_vecinos)
        p_may = 1.0 - p_min

        H = 0.0
        if p_min > 0.0:
            H -= p_min * np.log2(p_min)
        if p_may > 0.0:
            H -= p_may * np.log2(p_may)

        entropias[i] = H

    return entropias
```

Listing 6.2: Pureza mediante entropía de Shannon del vecindario

6.1.2. Fase 2 – Filtro de Densidad

El filtro de densidad mide qué proporción de los k vecinos de una semilla se encuentran dentro de un radio global u_{densidad}, calculado como el percentil `percentil_dist_densidad` de todas las distancias semilla→vecino.

```
def _calcular_umbral_global_desde_distancias(self,
    matriz_distancias, percentil):
    distancias_vector = matriz_distancias.reshape(-1)
    return float(np.percentile(distancias_vector,
        float(percentil)))

def _calcular_densidad_por_muestra(self, matriz_distancias,
    radio_densidad):
    """
    densidad_i = (# vecinos con dist <= radio_densidad) / k_vecinos
    Condicion: densidad_i >= umbral_densidad
    """
    n = matriz_distancias.shape[0]
    densidades = np.zeros(n, dtype=float)

    for i in range(n):
        vecinos_cercanos = sum(
            1 for d in matriz_distancias[i]
            if float(d) <= float(radio_densidad)
        )
        densidades[i] = vecinos_cercanos / float(self.k_vecinos)

    return densidades
```

Listing 6.3: Cálculo del umbral global de densidad y densidad por muestra

6.1.3. Fase 3 – Filtro de Riesgo

El filtro de riesgo cuantifica la presión de la clase mayoritaria sobre el vecindario de cada semilla. Se calcula la proporción de vecinos de la clase contraria que caen dentro de un radio de riesgo u_{riesgo}.

```
def _calcular_riesgo_por_muestra(
    self, y_binaria, matriz_indices_vecinos, matriz_distancias,
        radio_riesgo
):
    """
    riesgo_i = (# vecinos de clase 0 con dist <= radio_riesgo) /
        k_vecinos
    Condicion: riesgo_i <= umbral_riesgo
    """
    y_binaria = np.asarray(y_binaria)
    n = matriz_indices_vecinos.shape[0]
    riesgos = np.zeros(n, dtype=float)

    for i in range(n):
        indices = matriz_indices_vecinos[i]
        distancias = matriz_distancias[i]
        contrarios_cercanos = sum(
            1 for j, idx in enumerate(indices)
            if float(distancias[j]) <= float(radio_riesgo)
            and int(y_binaria[idx]) == 0
        )
        riesgos[i] = contrarios_cercanos / float(self.k_vecinos)

    return riesgos
```

Listing 6.4: Cálculo del riesgo por muestra (presión de clase mayoritaria)

6.1.4. Combinación de filtros y generación de muestras sintéticas

Los tres filtros se combinan con un `AND` lógico para obtener la máscara de semillas candidatas. Las muestras sintéticas se generan interpolando entre la semilla elegida y un vecino positivo aleatorio dentro del radio de densidad, utilizando un factor $\delta \in [0{,}4, 0{,}6]$ para posicionar el nuevo punto en la región intermedia (no demasiado cercano a ninguno de los dos extremos).

```
# Combinar los tres filtros
mascara_densidad  = densidades >= self.umbral_densidad
mascara_riesgo    = riesgos    <= self.umbral_riesgo
mascara_candidata = mascara_pureza & mascara_densidad &
    mascara_riesgo

# Generacion de muestra sintetica para cada iteracion
indice_semilla  = rng.choice(indices_locales_candidatas)
x_semilla       = X[indices_positivos[indice_semilla]]
vecino_elegido  = rng.choice(vecinos_positivos_validos)
x_vecino        = X[vecino_elegido]

delta   = rng.uniform(0.4, 0.6)   # interpolacion en zona
    intermedia
x_nueva = x_semilla + delta * (x_vecino - x_semilla)
```

Listing 6.5: Combinación de filtros y generación de muestras sintéticas

6.2. Pipeline de Sobremuestreo

El pipeline de sobremuestreo opera sobre el conjunto de entrenamiento ya escalado con `RobustScaler` (ajustado únicamente sobre el train). Se implementan dos funciones principales: una para las técnicas clásicas (SMOTE, BorderlineSMOTE, ADASYN) y otra para PC-SMOTE.

6.2.1. Sobremuestreo con técnicas clásicas

```python
from imblearn.over_sampling import SMOTE, BorderlineSMOTE, ADASYN

def generar_aumentaciones_clasicas_y_guardar(
    nombre_dataset, X_train, y_train, col_target,
    ruta_clasicos="../datasets/datasets_aumentados/resampler_clasicos/",
    overwrite=False, percentil_isolation_etiqueta=0.0
):
    """
    Aplica SMOTE, BorderlineSMOTE y ADASYN al conjunto de entrenamiento
    y guarda cada dataset aumentado como archivo CSV.
    """
    resamplers = [
        ("smote",           SMOTE(random_state=42)),
        ("borderlinesmote", BorderlineSMOTE(random_state=42, kind="borderline-1")),
        ("adasyn",          ADASYN(random_state=42)),
    ]

    n_original = len(X_train)

    for nombre_resampler, resampler in resamplers:
        try:
            X_res, y_res = resampler.fit_resample(X_train.values, y_train.values)
        except ValueError as e:
            print(f"  Advertencia: {nombre_resampler} no pudo generar muestras: {e}")
            continue

        cant_sinteticos = len(X_res) - n_original

        nombre_archivo = (
            f"{nombre_resampler}_{nombre_dataset}"
            f"_I{percentil_isolation_etiqueta}"
            f"_sg{cant_sinteticos}_train.csv"
        )
        path_salida = os.path.join(ruta_clasicos, nombre_archivo)
        pd.DataFrame(X_res, columns=list(X_train.columns)) \
          .assign(**{col_target: y_res}) \
          .to_csv(path_salida, index=False)
```

Listing 6.6: Pipeline de sobremuestreo con técnicas clásicas

6.2.2. Sobremuestreo con PC-SMOTE

```
def aumentar_dataset_pcsmote_y_guardar(
    nombre_dataset, X_train_base, y_train_base,
    percentil_radio_densidad, umbral_densidad,
    umbral_pureza_proporcion, umbral_riesgo, percentil_riesgo,
    criterio_pureza, percentil_entropia,
    col_target="target", percentil_isolation_etiqueta=0.0,
    ruta_pcsmote="../datasets/datasets_aumentados/"
):
    """
    Instancia PC-SMOTE con la configuracion indicada, aplica el
    resampling y guarda el dataset aumentado con un nombre que
    codifica todos los hiperparametros utilizados.
    """
    sampler = PCSMOTE(
        random_state=42,
        criterio_pureza=criterio_pureza,
        percentil_dist_densidad=percentil_radio_densidad,
        percentil_dist_riesgo=percentil_riesgo,
        percentil_entropia=percentil_entropia,
        umbral_densidad=umbral_densidad,
        umbral_pureza=umbral_pureza_proporcion,
        umbral_riesgo=umbral_riesgo,
        grado_iso=percentil_isolation_etiqueta
    )

    X_res, y_res = sampler.fit_resample(X_train_base, y_train_base)
    cant_sinteticos = len(X_res) - len(X_train_base)

    # El nombre del archivo codifica todos los hiperparametros
    # Ejemplo: pcs_ecoli_PRD80_PR40_CPprop_UD040_Upp050_UR045_I0_SV096_SG310_train.csv
    nombre_archivo = (
        f"pcs_{nombre_dataset}_"
        f"PRD{int(percentil_radio_densidad)}_"
        f"PR{int(percentil_riesgo)}_"
        f"CP{'ent' if criterio_pureza=='entropia' else 'prop'}_"
        f"UD{int(umbral_densidad*100):03d}_"
        f"UR{int(umbral_riesgo*100):03d}_"
        f"I{int(percentil_isolation_etiqueta)}_"
        f"SV{sampler.cantidad_semillas_candidatas:03d}_"
        f"SG{cant_sinteticos:03d}_train.csv"
    )
    path_salida = os.path.join(ruta_pcsmote, nombre_archivo)
    pd.DataFrame(X_res).assign(**{col_target: y_res}).to_csv(path_salida, index=False)
    return sampler
```

Listing 6.7: Pipeline de sobremuestreo con PC-SMOTE

6.3. Pipeline de Entrenamiento y Evaluación

El pipeline de evaluación realiza una búsqueda aleatoria de hiperparámetros (`RandomizedSearchCV`) con validación cruzada estratificada sobre el conjunto de entrenamiento aumentado, y luego evalúa el mejor modelo sobre el conjunto de prueba original (sin sobremuestreo).

6.3.1. Búsqueda de hiperparámetros y evaluación en test

```
from sklearn.model_selection import StratifiedKFold,
    RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import f1_score, balanced_accuracy_score,
    recall_score, make_scorer

RANDOM_STATE = 42
SCORING_REFIT = "f1_macro"
SCORING_MULTIPLE = {
    "f1_macro":            "f1_macro",
    "balanced_accuracy":   "balanced_accuracy",
    "recall_macro":        make_scorer(recall_score,
        average="macro"),
}

def construir_estimador_random_forest():
    estimador = Pipeline([
        ("classifier", RandomForestClassifier(
            random_state=RANDOM_STATE,
            n_jobs=1, bootstrap=True,
            n_estimators=150, max_features="sqrt",
        ))
    ])
    espacio_busqueda = {
        "classifier__n_estimators": [150, 300],
        "classifier__max_features": ["sqrt", "log2"],
    }
    return estimador, espacio_busqueda

def ejecutar_busqueda_y_evaluar(
    estimator, space, X_train, y_train, X_test, y_test,
    n_iter=4, n_folds=5
):
    """
    1) RandomizedSearchCV sobre X_train con validacion cruzada
        estratificada.
    2) Evaluacion del mejor estimador sobre X_test (datos
        originales, sin aumentar).
    Retorna metricas CV y Test: F1-macro, Balanced Accuracy,
        Recall macro.
    """
    cv = StratifiedKFold(n_splits=n_folds, shuffle=True,
        random_state=RANDOM_STATE)

    search = RandomizedSearchCV(
        estimator=estimator,
```

```
        param_distributions=space,
        n_iter=n_iter,
        scoring=SCORING_MULTIPLE,
        refit=SCORING_REFIT,
        cv=cv,
        random_state=RANDOM_STATE,
        n_jobs=1,
    )
    search.fit(X_train, y_train)

    # Metricas en validacion cruzada (mejor candidato)
    best_idx  = search.best_index_
    cv_f1     = search.cv_results_["mean_test_f1_macro"][best_idx]
    cv_bacc   = search.cv_results_["mean_test_balanced_accuracy"][best_idx]
    cv_recall = search.cv_results_["mean_test_recall_macro"][best_idx]

    # Metricas sobre el conjunto de test
    y_pred     = search.best_estimator_.predict(X_test)
    test_f1    = f1_score(y_test, y_pred, average="macro")
    test_bacc  = balanced_accuracy_score(y_test, y_pred)
    test_recall = recall_score(y_test, y_pred, average="macro")

    return {
        "mejores_params": search.best_params_,
        "cv":   {"f1": cv_f1,   "bacc": cv_bacc,   "recall_macro": cv_recall},
        "test": {"f1": test_f1, "bacc": test_bacc, "recall_macro": test_recall},
    }
```

Listing 6.8: Búsqueda aleatoria de hiperparámetros y evaluación en conjunto de test

6.4. Limpieza de Outliers con Isolation Forest

La clase `IsolationCleaner` aplica un `IsolationForest` de forma independiente por clase, eliminando aproximadamente el $p\%$ de muestras con peores *scores* de anomalía dentro de cada clase. Este preprocesamiento se realiza **únicamente sobre el conjunto de entrenamiento**, preservando íntegramente el conjunto de test.

```
from sklearn.ensemble import IsolationForest
import numpy as np

class IsolationCleaner:

    @staticmethod
    def limpiarOutliers(
        X, y,
        percentil_umbral: float = 5.0,   # elimina ~p% peor por clase
        n_estimators=200,
        contamination="auto",
        random_state=42,
        normalizar_scores: bool = False,
```

```
        devolver_info=False,
        verbose=True,
    ):
        """
        Aplica IsolationForest de forma independiente por clase.
        El umbral de corte se determina por percentil sobre el
        decision_function de cada clase (no por el label -1/1).
        El TEST nunca se incluye en este proceso.
        """
        X = np.asarray(X); y = np.asarray(y)
        keep_mask = np.zeros(len(y), dtype=bool)
        removed_total = 0

        for c in np.unique(y):
            idx = np.where(y == c)[0]
            if len(idx) < 10:          # clase muy pequena: conservar todo
                keep_mask[idx] = True
                continue

            iforest = IsolationForest(
                n_estimators=n_estimators,
                contamination=contamination,
                random_state=random_state
            )
            iforest.fit(X[idx])
            scores = iforest.decision_function(X[idx])  # >0: inlier

            umbral = float(np.percentile(scores, percentil_umbral))
            keep_local = (scores >= umbral)
            keep_mask[idx] = keep_local
            removed_total += int(np.sum(~keep_local))

        X_clean = X[keep_mask]; y_clean = y[keep_mask]

        if verbose:
            print(f"IF por clase (percentil={percentil_umbral}%): "
                  f"removidos {removed_total}, quedan {len(y_clean)}.")

        return (X_clean, y_clean, {"removed_total": removed_total}) \
               if devolver_info else (X_clean, y_clean)
```

Listing 6.9: Limpieza de outliers por clase con Isolation Forest

Bibliografía

[1] Carvalho, M., Pinho, A. J., and Brás, S. Resampling approaches to handle class imbalance: a review from a data perspective. *Journal of Big Data 12* (2025), 71. URL: https://doi.org/10.1186/s40537-025-01119-4.

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research 16* (2002), 321–357. URL: https://doi.org/10.1613/jair.953.

[3] Feng, Y., and Li, J. A novel αDistance Borderline-ADASYN-SMOTE algorithm for imbalanced data and its application in Alzheimer's disease classification based on Dense Convolutional Network. In *Journal of Physics: Conference Series* (2021), vol. 2031, IOP Publishing, p. 012046. URL: https://iopscience.iop.org/article/10.1088/1742-6596/2031/1/012046/pdf.

[4] Fernández, A., García, S., Galar, M., Prati, R. C., Krawczyk, B., and Herrera, F. *Learning from Imbalanced Data Sets*. Springer, 2018. URL: https://doi.org/10.1007/978-3-319-98074-4.

[5] Galar, M., Fernández, A., Barrenechea, E., Bustince, H., and Herrera, F. A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews) 42*, 4 (2012), 463–484. URL: https://doi.org/10.1109/TSMCC.2011.2161285.

[6] Han, H., Wang, W.-Y., and Mao, B.-H. Borderline-smote: a new over-sampling method in imbalanced data sets learning. In *Advances in Intelligent Computing: International Conference on Intelligent Computing, ICIC 2005* (2005), Springer, pp. 878–887. URL: https://doi.org/10.1007/11538059_91.

[7] He, H., Bai, Y., Garcia, E. A., and Li, S. Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks* (2008), IEEE, pp. 1322–1328. URL: https://doi.org/10.1109/IJCNN.2008.4633969.

[8] Kaggle. Telco customer churn. https://www.kaggle.com/datasets/blastchar/telco-customer-churn, 2018.

[9] Kaggle. Machine predictive maintenance classification. https://www.kaggle.com/datasets/shivamb/machine-predictive-maintenance-classification, 2020.

[10] Khorshidi, H. A., and Aickelin, U. A synthetic over-sampling method with minority and majority classes for imbalance problems. *Knowledge and Information Systems 67* (2025), 5965–5998. URL: https://doi.org/10.1007/s10115-025-02394-6.

[11] Lyu, J., Yang, J., Su, Z., and Zhu, Z. Ld-smote: A novel local density estimation-based oversampling method for imbalanced datasets. *Symmetry 17*, 2 (2025), 160. URL: https://doi.org/10.3390/sym17020160.

[12] Nasaruddin, N., Masseran, N., Idris, W. M. R., and Ul-Saufie, A. Z. A smote pca hdbscan approach for enhancing water quality classification in imbalanced datasets. *Scientific Reports 15* (2025), 13059. URL: https://doi.org/10.1038/s41598-025-97248-0.

[13] Park, H., and Kim, H. AR-ADASYN: angle radius-adaptive synthetic data generation approach for imbalanced learning. *Statistics and Computing 34*, 5 (2024), 166. URL: https://doi.org/10.1007/s11222-024-10479-5.

[14] Poddar, G. M., Patil, R. V., and Kumar N, S. Approaches to handle data imbalance problem in predictive machine learning models: A comprehensive review. *International Journal of Intelligent Systems and Applications in Engineering 12*, 21s (2024), 841–856. URL: https://www.academia.edu/117048787.

[15] Pradipta, G. A., Wardoyo, R., Musdholifah, A., and Sanjaya, I. N. H. Radius-SMOTE: A new oversampling technique of minority samples based on radius distance for learning from imbalanced data. *IEEE Access 9* (2021), 74763–74777. URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9431216.

[16] Qiu, F., Yang, F., Xiong, Y., Chen, T., Xiao, P., Zheng, W., Chen, X., and Zhou, K. Vs-smote: Leveraging high-value spaces to balance noise control and data diversity for class-imbalanced learning. *Neurocomputing* (2025). URL: https://papers.ssrn.com/sol3/Delivery.cfm?abstractid=5234376.

[17] Santos, M. S., Abreu, P. H., Japkowicz, N., Fernández, A., and Santos, J. A unifying view of class overlap and imbalance: Key concepts, multi-view panorama, and open avenues for research. *Information Fusion 89* (2023), 228–253. URL: https://sci2s.ugr.es/sites/default/files/ficherosPublicaciones/2997_2022-INFFUS-Unifying%20view%20overlap.pdf.

[18] Saputra, D. C. E., Sunat, K., and Ratnaningsih, T. Smote-mrs: A novel smote-multiresolution sampling technique for imbalanced distribution to improve prediction of anemia. *IEEE Access 12* (2024), 154675–154701. URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10721227.

[19] UCI Machine Learning Repository. Statlog (shuttle) data set. https://archive.ics.uci.edu/dataset/148/statlog+shuttle, 1993.

[20] UCI Machine Learning Repository. Communities and crime data set. https://archive.ics.uci.edu/dataset/183/communities+and+crime, 2009.

[21] Wang, J., and Awang, N. Ch-smote algorithm: A novel approach to improve random forest classification on class imbalanced datasets. *Journal of Computational Analysis and Applications 33*, 7 (2024), 1331–1349. URL: https://eudoxuspress.com/index.php/pub/article/view/1234.

[22] Wang, J., and Awang, N. A novel synthetic minority oversampling technique for multiclass imbalance problems. *IEEE Access 13* (2025). URL: https://www.researchgate.net/publication/387809455.